

Python Interview Preparation Guide

Data structures, casting, file handling, error handling, OOP, APIs and pagination, logging, os / sys / argparse, data engineering patterns and medium-level coding problems

- 1-4 Core basics, operators, type casting, strings, regex
- 5 Lists, tuples, sets, dicts, collections, heapq, bisect, complexity
- 6-8 Control statements, functions, *args / **kwargs, generators, decorators, context managers
- 9 Exception handling: try / except / else / finally, custom exceptions, retries
- 10-13 File read / write / edit, CSV / JSON, os, sys, pathlib, argparse, logging
- 14 Object-oriented programming with full examples and cheat sheet
- 15 REST APIs: requests, auth, retries, four pagination styles, ETL example
- 16 pandas, DB-API, Snowflake connector, migration validation, concurrency, testing
- 17 DuckDB: connections, file queries, SQL features, pandas integration, UDFs, reconciliation
- 18 Commonly used built-in and standard library functions with explanations
- 19-20 Coding interview patterns (medium level) and rapid-fire Q&A
- 21 Completeness check for data engineering, what to study next, 7-day plan

How to use: run every snippet in a Python REPL or notebook, change the inputs, and explain each one aloud. Comments in green show expected output. Code targets Python 3.10+ (a few notes mention 3.11 features). Network, database and Snowflake examples need real credentials or endpoints; the fake-API demo in section 15 runs offline.

Table of Contents

1. Core Basics and Data Types	5
1.1 Variables, dynamic typing, mutability	5
1.2 Mutable vs immutable, is vs ==, copying	5
1.3 Truthiness	6
1.4 Input, output, comments, naming	6
2. Operators	6
2.1 Arithmetic	6
2.2 Comparison, logical, chained	6
2.3 Assignment, identity, membership	7
2.4 Bitwise operators	7
2.5 Ternary, precedence, float precision	7
3. Type Casting (Type Conversion)	8
3.1 Implicit vs explicit	8
3.2 Core conversions	8
3.3 Errors and safe casting	8
3.4 Dates and numbers as text (very common in data work)	8
4. Strings	9
4.1 Indexing, slicing, immutability	9
4.2 Important string methods	9
4.3 Formatting	9
4.4 Classic string problems	9
4.5 Regular expressions (re)	10
5. Data Structures	10
5.1 Lists	10
5.2 Tuples	11
5.3 Sets	11
5.4 Dictionaries	11
5.5 Comprehensions	12
5.6 Copying and nested structures	12
5.7 List of dicts (the "rows" of data engineering)	13
5.8 collections module	13
5.9 heapq, bisect, stack, queue	13
5.10 Complexity cheat sheet	14
6. Control Statements	14
6.1 if / elif / else and match-case	14
6.2 for loops, range, enumerate, zip	14
6.3 while, break, continue, else on loops	15
6.4 Pattern and classic loop problems	15
7. Functions and Argument Handling	16
7.1 Defining functions, defaults, return values	16
7.2 The mutable default argument trap	16
7.3 *args and **kwargs	16
7.4 Positional-only and keyword-only parameters	17
7.5 Scope (LEGB), global, nonlocal	17
7.6 Lambda, map, filter, reduce	17
7.7 Type hints and docstrings	17
7.8 Recursion and memoization	18
8. Iterators, Generators, Decorators, Context Managers	18
8.1 Iterables vs iterators	18
8.2 Generators (memory-efficient, lazy)	18
8.3 itertools highlights	19
8.4 Decorators	19
8.5 Context managers (with statement)	19
9. Exception Handling	20

9.1 try / except / else / finally	20
9.2 Common built-in exceptions	20
9.3 raise, custom exceptions, chaining, assert	21
9.4 Patterns used in real pipelines	21
10. File Handling: Read, Write, Edit	22
10.1 open() modes	22
10.2 Reading and writing text	22
10.3 Editing a file (the edit process)	23
10.4 CSV	23
10.5 JSON, JSON Lines, pickle	24
10.6 pathlib, shutil, temp files, compression	24
10.7 Big-file strategies	25
11. os, sys, and Related Standard Library	25
11.1 os module	25
11.2 sys module	26
11.3 subprocess, glob, datetime, time	26
12. Command-Line Arguments: sys.argv and argparse	27
12.1 Raw sys.argv	27
12.2 argparse	27
13. Logging	28
13.1 Why logging instead of print	28
13.2 Quick setup and usage	28
13.3 Handlers, formatters, rotation, dictConfig	28
14. Object-Oriented Programming (OOP)	29
14.1 Classes, objects, __init__, instance vs class attributes	29
14.2 Instance, class, and static methods	29
14.3 Encapsulation: public, __protected__, __private, properties	30
14.4 Inheritance, super(), method overriding	31
14.5 Multiple inheritance and MRO	31
14.6 Polymorphism, duck typing, abstract base classes	31
14.7 Special (dunder) methods	32
14.8 dataclasses, enums, __slots__	32
14.9 Composition, design patterns, class-based tools	33
14.10 Full worked example: inventory system	34
14.11 OOP interview cheat sheet	35
15. Working with APIs (REST) and Pagination	35
15.1 HTTP and REST basics	35
15.2 requests library: GET, POST, params, headers	35
15.3 Authentication patterns	36
15.4 Session, retries and rate limits	36
15.5 Pagination: the four common styles	37
15.6 End-to-end extract -> transform -> load example	38
15.7 Incremental loads, idempotency, JSON to DataFrame	39
15.8 Concurrency for API calls	39
15.9 Testing API code without the network	39
16. Python for Data Engineering	40
16.1 pandas essentials	40
16.2 Databases with sqlite3 (DB-API pattern used by every driver)	40
16.3 Snowflake connector and bulk loading (data migration pattern)	40
16.4 Migration and validation patterns	41
16.5 Concurrency: threads vs processes vs asyncio	42
16.6 Project hygiene: entry point, config, tests, environments	42
17. DuckDB: Connections, SQL, and Python Functions	43
17.1 What DuckDB is and when to use it	43
17.2 Connecting	43
17.3 Running queries and getting results	44
17.4 Reading files directly (CSV, Parquet, JSON)	44
17.5 Writing files and exporting	45

17.6 pandas, Arrow and Polars integration	45
17.7 Relational API (method chaining instead of SQL)	45
17.8 SQL features that make DuckDB pleasant (and are asked about)	46
17.9 Tables, constraints, upserts, transactions	46
17.10 Extensions, remote files, attaching other databases	47
17.11 Settings, performance, and out-of-core processing	47
17.12 Python UDFs, cursors, threads, and errors	48
17.13 Date, string and conversion functions you will use daily	48
17.14 End-to-end example: reconcile a source extract with a target (data migration)	49
17.15 DuckDB interview cheat sheet	49
18. Commonly Used Python Functions (with explanations)	50
18.1 Built-in functions: numbers and aggregation	50
18.2 Built-ins: iteration and functional	50
18.3 Built-ins: types, objects and introspection	50
18.4 String / list / dict method quick recap	51
18.5 Standard library functions you will use constantly	51
18.6 One-liners worth memorising	52
19. Coding Interview Patterns (Medium Level)	52
19.1 Hash map, counting, grouping	52
19.2 Two pointers and sliding window	53
19.3 Sorting, searching, intervals, matrices	53
19.4 Stack, queue, linked list, tree, graph	54
19.5 Recursion, backtracking, dynamic programming	55
19.6 Design a class: LRU cache and rate limiter	55
19.7 Data-engineering style coding questions	56
19.8 Big-O quick reference	57
20. Interview Questions and Short Answers	57
20.1 Python fundamentals	57
20.2 Data engineering and API scenario questions	58
21. Completeness Check and What To Add Next	59
21.1 What this guide covers	59
21.2 Verdict for Python data engineering work	59
21.3 Study plan and last-minute checklist	59

1. Core Basics and Data Types

1.1 Variables, dynamic typing, mutability

Python is dynamically typed: a **name** is a label bound to an **object**; the type lives on the object, not the name. Everything is an object.

```
x = 10                # int
x = "ten"            # same name, now bound to a str (allowed)
a, b, c = 1, 2.5, "hi" # multiple assignment
a, b = b, a           # swap without a temp variable
x = y = 0            # chained assignment: both names -> same object
first, *rest = [1, 2, 3, 4] # extended unpacking: first=1, rest=[2, 3, 4]
*head, last = [1, 2, 3, 4] # head=[1, 2, 3], last=4
print(type(x), id(x))    # <class 'int'>, memory identity in CPython
```

Type	Example	Mutable	Ordered	Notes
int	42, 1_000_000	No	-	Arbitrary precision
float	3.14, 1e-3	No	-	64-bit IEEE 754
complex	2+3j	No	-	Rare in interviews
bool	True, False	No	-	Subclass of int (<code>True == 1</code>)
str	"abc"	No	Yes	Unicode text
bytes	b"abc"	No	Yes	Raw bytes
list	[1, 2]	Yes	Yes	Dynamic array
tuple	(1, 2)	No	Yes	Hashable if items hashable
set	{1, 2}	Yes	No	Unique, hash based
frozenset	frozenset({1})	No	No	Hashable set
dict	{"a": 1}	Yes	Insertion (3.7+)	Key -> value hash map
NoneType	None	No	-	Singleton "no value"

1.2 Mutable vs immutable, is vs ==, copying

```
a = [1, 2, 3]
b = a                # NOT a copy: both names point to the same list
b.append(4)
print(a)             # [1, 2, 3, 4] <- a changed too

t = (1, 2, 3)
# t[0] = 9           # TypeError: tuple does not support item assignment

x = [1, 2]; y = [1, 2]
print(x == y)        # True   (equal values)
print(x is y)        # False  (different objects)
print(x is x)        # True

# Always test None with "is", never "=="
value = None
if value is None:
    print("no value")

# Small-int caching (CPython detail, do not rely on it)
p = 256; q = 256
print(p is q)        # True
```

```
import copy
orig = [[1, 2], [3, 4]]
shallow = orig.copy()    # or list(orig) or orig[:]
deep = copy.deepcopy(orig)
orig[0].append(99)
print(shallow) # [[1, 2, 99], [3, 4]] inner list is shared
print(deep)    # [[1, 2], [3, 4]] fully independent
```

1.3 Truthiness

Falsy values: `None`, `False`, `0`, `0.0`, `""`, `[]`, `()`, `{}`, `set()`, `range(0)`. Everything else is truthy. Prefer `if items:` over `if len(items) > 0:`.

```
name = ""
print(bool(name))          # False
data = []
if not data:
    print("empty list")
```

1.4 Input, output, comments, naming

```
name = input("Name: ")      # always returns str
age = int(input("Age: "))    # cast explicitly
print("a", "b", sep="-", end="\n") # a-b!
print(f"{name} is {age} years old") # f-string (3.6+)
# Single-line comment
"""Docstring / multi-line string, used as documentation."""
```

Naming (PEP 8): `snake_case` for variables/functions, `PascalCase` for classes, `UPPER_CASE` for constants, `_private` by convention, `__mangled` for name-mangling, `__dunder__` for special methods.

2. Operators

2.1 Arithmetic

```
print(7 + 2, 7 - 2, 7 * 2) # 9 5 14
print(7 / 2)               # 3.5 true division always returns float
print(7 // 2)              # 3 floor division
print(-7 // 2)             # -4 floors toward negative infinity (trap!)
print(7 % 3)               # 1 modulo
print(-7 % 3)              # 2 result takes the sign of the divisor
print(2 ** 10)             # 1024 power
print(divmod(17, 5))        # (3, 2) quotient and remainder together
print(abs(-3), round(2.675, 2), round(3.5), round(4.5)) # 3 2.67 4 4 (banker's rounding)
```

2.2 Comparison, logical, chained

```
x = 5
print(1 < x < 10)          # True chained comparison
print(x == 5, x != 5, x >= 5)
print(True and False, True or False, not True)

# and / or return an OPERAND, not just True/False
print("" or "default")    # default (first truthy)
print("a" and "b")        # b (last if all truthy)
print(0 and 10)           # 0 (first falsy)

# Short-circuit: right side is skipped when result is known
d = {}
if "k" in d and d["k"] > 3: # safe, d["k"] never evaluated
    pass
```

2.3 Assignment, identity, membership

```
n = 10
n += 5; n -= 3; n *= 2; n /= 4; n **= 2; n %= 7    # augmented assignment

print(3 in [1, 2, 3], "a" not in "cat")           # True False
print("k" in {"k": 1})                           # True (dict membership checks KEYS)
print(2 in {1, 2, 3})                             # 0(1) for set/dict, 0(n) for list/tuple
a = b = [1]
print(a is b, a is not None) # True True

# Walrus operator := (3.8+) assigns inside an expression
data = [1, 2, 3, 4]
if (n := len(data)) > 3:
    print(f"long list: {n}")
```

2.4 Bitwise operators

```
a, b = 0b1100, 0b1010      # 12, 10
print(a & b)                # 8  AND  (0b1000)
print(a | b)                # 14 OR   (0b1110)
print(a ^ b)                # 6  XOR  (0b0110)
print(~a)                   # -13 NOT  (-(a+1))
print(a << 2, a >> 2)        # 48 3  shift = multiply / divide by 2**n
print(bin(a), hex(255), oct(8)) # 0b1100 0xff 0o10
print(bool(a & 1))           # odd/even check via lowest bit -> False (12 is even)
```

2.5 Ternary, precedence, float precision

```
age = 20
status = "adult" if age >= 18 else "minor"        # ternary expression

# Precedence high -> low: **, unary +~ , * / // % , + - , << >> , & , ^ , | ,
# comparisons/in/is , not , and , or , if-else , :=
print(2 + 3 * 4 ** 2)      # 50
print((2 + 3) * 4)         # 20 parentheses win

print(0.1 + 0.2)           # 0.30000000000000004
print(0.1 + 0.2 == 0.3)    # False
import math
print(math.isclose(0.1 + 0.2, 0.3))    # True compare floats like this
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2")) # 0.3 use Decimal for money
```

3. Type Casting (Type Conversion)

3.1 Implicit vs explicit

Implicit: Python widens automatically (`int + float -> float`, `bool + int -> int`). Explicit: you call a constructor such as `int()`, `str()`.

```
print(1 + 2.0)          # 3.0      int promoted to float
print(True + 1)         # 2
print(type(3 / 1))      # <class 'float'>
```

3.2 Core conversions

```
print(int("42"), int(3.99), int(-3.99))    # 42 3 -3   (truncates toward zero)
print(int("101", 2), int("ff", 16))        # 5 255    (string in base n)
print(float("3.5"), float("1e3"), float(7)) # 3.5 1000.0 7.0
print(str(99), str(3.0), str([1, 2]), str(None)) # '99' '3.0' '[1, 2]' 'None'
print(bool(0), bool("0"), bool("False"), bool([])) # False True True False (trap!)

print(list("abc"))          # ['a', 'b', 'c']
print(list((1, 2)), list({3, 4}), list({"a": 1})) # [1, 2] [3, 4] ['a']
print(tuple([1, 2]), set([1, 1, 2]))        # (1, 2) {1, 2}
print(dict([("a", 1), ("b", 2)]))          # {'a': 1, 'b': 2}
print(dict(zip(["x", "y"], [1, 2])))       # {'x': 1, 'y': 2}
print(list(range(3)), list(map(int, "123"))) # [0, 1, 2] [1, 2, 3]
print(ord("A"), chr(97))                  # 65 a
print(".".join(["a", "b"]), "a,b".split(",")) # 'a,b' ['a', 'b']
print(bytes("hi", "utf-8"), b"hi".decode("utf-8")) # b'hi' hi
print(frozenset([1, 2]), complex(1, 2))
```

3.3 Errors and safe casting

```
# int("3.5") -> ValueError ; int(None) -> TypeError ; int("") -> ValueError
print(int(float("3.5")))    # 3   go through float first

def to_int(value, default=None):
    """Convert to int, return default if not possible."""
    try:
        return int(value)
    except (ValueError, TypeError):
        return default

print(to_int("12"), to_int("abc", 0), to_int(None))    # 12 0 None

def to_bool(s):
    return str(s).strip().lower() in {"true", "1", "yes", "y", "t"}
print(to_bool("Yes"), to_bool("0"))    # True False
```

3.4 Dates and numbers as text (very common in data work)

```
from datetime import datetime
d = datetime.strptime("2026-09-24 14:30", "%Y-%m-%d %H:%M") # str -> datetime
print(d.strftime("%d/%m/%Y"))                               # datetime -> str: 24/09/2026
print(int(d.timestamp()))                                    # datetime -> epoch seconds
print(datetime.fromtimestamp(0).year)                        # epoch -> datetime
print(f"{1234567.891:.2f}")                                  # 1,234,567.89 number -> formatted string
print(int("1,234".replace(",", "")))                        # 1234 clean before casting
```

4. Strings

4.1 Indexing, slicing, immutability

```
s = "Python"
print(s[0], s[-1])          # P n
print(s[1:4])              # yth      [start:stop) stop excluded
print(s[:2], s[2:])        # Py thon
print(s[::2])              # Pto      step 2
print(s[::-1])             # nohtyP   reverse
# s[0] = "J"               # TypeError (immutable): build a new string
s = "J" + s[1:]            # Jython
print(len(s), "th" in s)   # 6 True
```

4.2 Important string methods

```
t = " Hello, World "
print(t.strip(), t.lstrip(), t.rstrip()) # remove whitespace (or given chars)
print(t.lower(), t.upper(), t.title(), t.capitalize(), t.swapcase())
print(t.replace("l", "L", 1))           # replace first occurrence only
print("a,b,c".split(","))               # ['a', 'b', '', 'c']
print("a b c".split())                  # ['a', 'b', 'c'] (any whitespace)
print("k=v=w".split("=", 1))            # ['k', 'v=w']      maxsplit
print("-".join(["2026", "09", "24"]))    # 2026-09-24
print("abc".startswith("ab"), "abc".endswith("c"))
print("banana".find("na"), "banana".rfind("na"), "banana".count("a")) # 2 4 3
# find returns -1 when missing; index raises ValueError
print("42".isdigit(), "ab".isalpha(), "a1".isalnum(), " ".isspace())
print("7".zfill(3), "ab".center(6, "*"), "ab".ljust(4, ".") + "|") # 007 **ab** ab..|
print("a\nb\nc".splitlines())           # ['a', 'b', 'c']
print("x".join("abc"))                  # axbxc
print("hello world".partition(" "))     # ('hello', ' ', 'world')
```

4.3 Formatting

```
name, score, pi = "Asha", 91.456, 3.14159
print(f"{name} scored {score:.1f}")      # Asha scored 91.5
print(f"{name:>10}|{name:<10}|{name:^10}|") # right / left / center aligned
print(f"{42:05d} {255:x} {255:08b} {0.256:1%}") # 00042 ff 11111111 25.6%
print(f"{1_000_000:,}")                 # 1,000,000
print(f"{name=}")                       # name='Asha' (3.8+ debug form)
print("{} is {}".format("x", 1), "%s=%d" % ("n", 3)) # older styles
print(r"raw \n string", "tab\there")    # raw strings ignore escapes
```

4.4 Classic string problems

```
from collections import Counter

def is_palindrome(s):
    cleaned = [c.lower() for c in s if c.isalnum()]
    return cleaned == cleaned[::-1]

def is_anagram(a, b):
    return Counter(a.replace(" ", "").lower()) == Counter(b.replace(" ", "").lower())

def first_unique_char(s):
    counts = Counter(s)
    for i, ch in enumerate(s):
        if counts[ch] == 1:
            return i
    return -1

def compress(s):
    # "aaabbc" -> "a3b2c1"
    if not s:
```

```

    return ""
    out, count = [], 1
    for prev, cur in zip(s, s[1:]):
        if cur == prev:
            count += 1
        else:
            out.append(f"{prev}{count}")
            count = 1
    out.append(f"{s[-1]}{count}")
    return "".join(out)

print(is_palindrome("A man, a plan, a canal: Panama")) # True
print(is_anagram("listen", "silent"))                  # True
print(first_unique_char("swiss"))                      # 1 ('w')
print(compress("aaabbc"))                             # a3b2c1
print(" ".join(w[::-1] for w in "hello big world".split())) # olleh gib dlrow
print(" ".join("hello big world".split()[::-1]))       # world big hello
vowels = sum(ch in "aeiou" for ch in "education")      # 5

```

4.5 Regular expressions (re)

```

import re
text = "Order 123 shipped on 2026-09-24 to bob@mail.com"
print(re.findall(r"\d+", text)) # ['123', '2026', '09', '24']
m = re.search(r"(\d{4})-(\d{2})-(\d{2})", text)
print(m.group(0), m.groups()) # 2026-09-24 ('2026', '09', '24')
print(re.sub(r"s+", " ", "a b c")) # a b c
print(re.match(r"^\w+@\w+\.\w+$", "bob@mail.com") is not None) # True
pat = re.compile(r"(?P<key>\w+)= (?P<val>\w+)") # compile when reused; named groups
print(pat.search("mode=fast").groupdict()) # {'key': 'mode', 'val': 'fast'}
print(re.split(r"[;,]\s*", "a; b,c")) # ['a', 'b', 'c']

```

match anchors at the start, **search** scans anywhere, **fullmatch** needs the entire string, **findall** returns all matches, **finditer** yields match objects.

5. Data Structures

5.1 Lists

Ordered, mutable, allow duplicates and mixed types. Backed by a dynamic array: index $O(1)$, append $O(1)$ amortised, insert/remove in the middle $O(n)$, membership $O(n)$.

```

nums = [5, 3, 8, 1]
nums.append(9) # add at end -> [5, 3, 8, 1, 9]
nums.insert(1, 7) # insert at index -> [5, 7, 3, 8, 1, 9]
nums.extend([2, 2]) # add many -> [..., 2, 2]
nums += [0] # same as extend
print(nums.pop()) # remove+return last -> 0
print(nums.pop(0)) # remove+return index 0 -> 5
nums.remove(2) # remove FIRST value 2 (ValueError if absent)
del nums[0] # delete by index
print(nums.index(8), nums.count(2)) # position of first 8, occurrences of 2
nums.reverse() # in place
nums.sort() # in place, returns None
nums.sort(reverse=True)
print(sorted(nums)) # returns a NEW list, original untouched
nums.clear()

a = [1, 2, 3, 4, 5, 6]
print(a[1:4], a[-2:], a[::2], a[::-1]) # [2,3,4] [5,6] [1,3,5] [6,5,4,3,2,1]
a[1:3] = [20, 30, 40] # slice assignment can change length
print(len(a), max(a), min(a), sum(a))
print([0] * 3) # [0, 0, 0]
grid = [[0] * 3 for _ in range(2)] # correct 2D list
bad = [[0] * 3] * 2 # WRONG: both rows are the same list
bad[0][0] = 1
print(bad) # [[1, 0, 0], [1, 0, 0]]

```

```
# Sorting with keys, stability, multi-key sort
people = [("Ann", 30), ("Bob", 25), ("Cy", 30)]
print(sorted(people, key=lambda p: p[1]))           # by age (stable: Ann before Cy)
print(sorted(people, key=lambda p: (-p[1], p[0])))  # age desc, then name asc
print(sorted(["b", "A", "c"], key=str.lower))      # ['A', 'b', 'c']
words = ["pear", "fig", "apple"]
print(sorted(words, key=len))                      # ['fig', 'pear', 'apple']
```

5.2 Tuples

Ordered, immutable, hashable (if contents are), so they work as dict keys and set members. Used for fixed records, multiple return values, unpacking.

```
pt = (3, 4)
single = (5,)           # trailing comma makes a 1-tuple; (5) is just int
x, y = pt               # unpacking
print(pt[0], pt + (5,), pt * 2, len(pt), pt.count(3), pt.index(4))
def min_max(items):
    return min(items), max(items)      # returns a tuple
lo, hi = min_max([4, 1, 9])
cache = {(0, 0): "origin", (1, 2): "A"} # tuple as dict key
from collections import namedtuple
Point = namedtuple("Point", "x y")
p = Point(1, 2)
print(p.x, p[1], p._asdict())          # 1 2 {'x': 1, 'y': 2}
```

5.3 Sets

Unordered collection of unique hashable items. Add, remove, membership are O(1) on average. Great for de-duplication and comparisons between datasets.

```
s = {1, 2, 3}
empty = set()           # {} creates a dict, not a set!
s.add(4); s.discard(10)  # discard never raises; remove(10) raises KeyError
s.update([5, 6])
print(s.pop())          # removes an arbitrary element
a, b = {1, 2, 3, 4}, {3, 4, 5}
print(a | b, a.union(b))      # union      {1,2,3,4,5}
print(a & b, a.intersection(b)) # intersection {3,4}
print(a - b, a.difference(b))  # in a not b {1,2}
print(a ^ b, a.symmetric_difference(b)) # in exactly one {1,2,5}
print(a <= b, {3} <= a, a.isdisjoint({9})) # subset / superset / no overlap
print(list(dict.fromkeys([3, 1, 3, 2, 1]))) # [3, 1, 2] de-dupe KEEPING order
print(len(set("mississippi")))             # 4
# Data-engineering use: find rows missing from target
source_ids, target_ids = {1, 2, 3, 4}, {1, 2, 4}
print(source_ids - target_ids)              # {3} missing in target
```

5.4 Dictionaries

Key -> value hash map. Keys must be hashable (str, int, tuple), average O(1) get/set/delete. Keeps insertion order (3.7+).

```
d = {"name": "Asha", "age": 30}
d["city"] = "Pune"        # add / update
print(d["name"], d.get("zip"), d.get("zip", "N/A")) # get never raises KeyError
print(d.setdefault("tags", [])) # returns value, inserts default if missing
d.update({"age": 31, "role": "DE"}) # merge / overwrite
print(d.pop("role"), d.pop("nope", None)) # pop with default avoids KeyError
del d["city"]
print(d.keys(), d.values(), d.items()) # live views
for k, v in d.items():
    print(k, v)
print("age" in d, len(d))
merged = {**d, **{"age": 40}} # unpack merge (later wins)
merged2 = d | {"x": 1}        # 3.9+ union operator
print(dict(sorted(d.items(), key=lambda kv: kv[0]))) # sort by key
inv = {v: k for k, v in {"a": 1, "b": 2}.items()}    # invert -> {1: 'a', 2: 'b'}
```

```

# Counting and grouping patterns (asked very often)
words = ["apple", "bob", "cat", "apple", "bob", "apple"]

freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1      # manual count
print(freq)                          # {'apple': 3, 'bob': 2, 'cat': 1}

groups = {}
for w in ["ant", "bee", "art", "bat"]:
    groups.setdefault(w[0], []).append(w)
print(groups)                        # {'a': ['ant', 'art'], 'b': ['bee', 'bat']}

# Nested dict access safely
cfg = {"db": {"host": "x", "port": 5432}}
print(cfg.get("db", {}).get("port")) # 5432
print(cfg.get("cache", {}).get("ttl")) # None (no KeyError)
# max by value
scores = {"a": 3, "b": 9, "c": 5}
print(max(scores, key=scores.get))    # b

```

5.5 Comprehensions

```

squares = [n * n for n in range(6)]          # [0, 1, 4, 9, 16, 25]
evens = [n for n in range(10) if n % 2 == 0] # filter
labels = ["even" if n % 2 == 0 else "odd" for n in range(3)] # if-else goes BEFORE for
pairs = [(i, j) for i in range(2) for j in range(2)] # nested loops
flat = [x for row in [[1, 2], [3], [4, 5]] for x in row] # flatten: [1,2,3,4,5]
sq_map = {n: n * n for n in range(4)}        # dict comprehension
uniq_len = {len(w) for w in ["a", "bb", "cc"]} # set comprehension {1, 2}
gen = (n * n for n in range(1_000_000))      # generator: lazy, O(1) memory
print(sum(n * n for n in range(10)))        # 285, no brackets needed

```

5.6 Copying and nested structures

```

import copy
rec = {"id": 1, "tags": ["a", "b"]}
shallow = rec.copy()                # **rec or dict(rec) also shallow
deep = copy.deepcopy(rec)
rec["tags"].append("c")
print(shallow["tags"], deep["tags"]) # ['a', 'b', 'c'] ['a', 'b']

# Flatten arbitrarily nested lists
def flatten(items):
    for it in items:
        if isinstance(it, (list, tuple)):
            yield from flatten(it)
        else:
            yield it
print(list(flatten([1, [2, [3, [4]], 5])))) # [1, 2, 3, 4, 5]

# Flatten nested dict (common when handling JSON from APIs)
def flatten_dict(d, parent="", sep="."):
    out = {}
    for k, v in d.items():
        key = f"{parent}{sep}{k}" if parent else k
        if isinstance(v, dict):
            out.update(flatten_dict(v, key, sep))
        else:
            out[key] = v
    return out
print(flatten_dict({"a": 1, "b": {"c": 2, "d": {"e": 3}}})) # {'a':1,'b.c':2,'b.d.e':3}

```

5.7 List of dicts (the "rows" of data engineering)

```
rows = [
    {"id": 1, "dept": "IT", "sal": 100},
    {"id": 2, "dept": "HR", "sal": 80},
    {"id": 3, "dept": "IT", "sal": 120},
]
print(sorted(rows, key=lambda r: r["sal"], reverse=True)[0])      # highest paid
print([r["id"] for r in rows if r["dept"] == "IT"])              # filter+project
print(sum(r["sal"] for r in rows) / len(rows))                   # average
by_dept = {}
for r in rows:
    by_dept.setdefault(r["dept"], []).append(r["sal"])
print({k: sum(v) / len(v) for k, v in by_dept.items()})           # group-by average
lookup = {r["id"]: r for r in rows}                              # index by key: O(1) joins
print(lookup[3]["dept"])                                         # IT
dedup = list({r["id"]: r for r in rows}.values())                # de-dupe by id (last wins)
```

5.8 collections module

```
from collections import Counter, defaultdict, deque, OrderedDict, ChainMap

c = Counter("mississippi")
print(c)                  # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
print(c.most_common(2))   # [('i', 4), ('s', 4)]
c.update("ss"); print(c["s"]) # 6 ; missing keys return 0, not KeyError
print(Counter([1, 2, 2]) + Counter([2, 3])) # Counter({2: 3, 1: 1, 3: 1})

dd = defaultdict(list)    # missing key -> factory() result
dd["a"].append(1); dd["a"].append(2)
print(dict(dd))            # {'a': [1, 2]}
cnt = defaultdict(int); cnt["x"] += 1 # counting without .get
nested = defaultdict(lambda: defaultdict(int)); nested["u"]["v"] += 1

dq = deque([1, 2, 3], maxlen=3) # O(1) at both ends; maxlen drops oldest
dq.append(4); dq.appendleft(0)
print(dq, dq.pop(), dq.popleft())
dq.rotate(1)

od = OrderedDict(a=1, b=2); od.move_to_end("a") # ordering helpers (LRU pattern)
defaults = ChainMap({"env": "dev"}, {"env": "prod", "debug": False})
print(defaults["env"], defaults["debug"])      # dev False (first mapping wins)
```

5.9 heapq, bisect, stack, queue

```
import heapq, bisect
h = []
for n in [5, 1, 8, 3]:
    heapq.heappush(h, n) # min-heap
print(heapq.heappop(h)) # 1
print(heapq.nlargest(2, [5, 1, 8, 3]), heapq.nsmallest(2, [5, 1, 8, 3])) # [8,5] [1,3]
lst = [4, 2, 9]
heapq.heapify(lst) # in-place O(n)
# max-heap trick: push negatives
mh = []; heapq.heappush(mh, -10); print(-heapq.heappop(mh)) # 10
# top-k by key
print(heapq.nlargest(2, [("a", 3), ("b", 9), ("c", 5)], key=lambda t: t[1]))

sorted_list = [1, 3, 5, 7]
print(bisect.bisect_left(sorted_list, 5), bisect.bisect_right(sorted_list, 5)) # 2 3
bisect.insort(sorted_list, 4) # keeps list sorted -> [1, 3, 4, 5, 7]

stack = [] # LIFO with list
stack.append(1); stack.append(2); print(stack.pop()) # 2
from collections import deque
queue = deque() # FIFO: use deque, NOT list.pop(0) which is O(n)
queue.append("a"); queue.append("b"); print(queue.popleft()) # a
from queue import Queue, PriorityQueue # thread-safe versions
```

5.10 Complexity cheat sheet

Operation	list	dict / set	deque	Notes
Index / lookup	$O(1)$	$O(1)$ avg	$O(n)$ middle	dict lookup by key
Append / add	$O(1)$ amortised	$O(1)$ avg	$O(1)$	
Insert / delete at front	$O(n)$	-	$O(1)$	use deque
Membership (<code>in</code>)	$O(n)$	$O(1)$ avg	$O(n)$	convert list to set for many lookups
Sort	$O(n \log n)$	-	-	Timsort, stable
Iterate	$O(n)$	$O(n)$	$O(n)$	
Slice	$O(k)$	-	-	creates a copy

6. Control Statements

6.1 if / elif / else and match-case

```
score = 78
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
else:
    grade = "C"
print(grade)                                # B

# Python 3.10+ structural pattern matching
def handle(cmd):
    match cmd.split():
        case ["quit"]:
            return "bye"
        case ["load", filename]:
            return f"loading {filename}"
        case ["go", ("north" | "south") as direction]:
            return f"going {direction}"
        case [first, *rest]:
            return f"{first} with {len(rest)} args"
        case _:
            return "empty"
print(handle("load data.csv"))              # loading data.csv
```

6.2 for loops, range, enumerate, zip

```
for i in range(3):
    print(i, end=" ")    # 0 1 2
for i in range(2, 10, 3):
    print(i, end=" ")    # 2 5 8 (start, stop, step)
for i in range(5, 0, -1):
    print(i, end=" ")    # 5 4 3 2 1
for ch in "abc":
    print(ch, end=" ")    # iterate any iterable

fruits = ["apple", "kiwi"]
for idx, f in enumerate(fruits, start=1):
    print(idx, f)         # index + value
names, ages = ["A", "B", "C"], [30, 25]
for n, a in zip(names, ages):
    print(n, a)           # stops at the shortest
                           # A 30 / B 25
from itertools import zip_longest
print(list(zip_longest(names, ages, fillvalue=0))) # [('A',30), ('B',25), ('C',0)]
for k, v in {"x": 1}.items():
    print(k, v)
for i in reversed(range(3)):
    pass
for a, (b, c) in [(1, (2, 3))]:
    print(a, b, c)        # nested unpacking
```

6.3 while, break, continue, else on loops

```
n = 0
while n < 5:
    n += 1
    if n == 2:
        continue          # skip rest of this iteration
    if n == 4:
        break              # leave the loop entirely
    print(n, end=" ")      # 1 3
print()

# for/while ... else: else runs only if the loop was NOT ended by break
for x in [1, 3, 5]:
    if x % 2 == 0:
        print("found even"); break
else:
    print("no even numbers")          # printed

def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True
print([p for p in range(20) if is_prime(p)]) # [2, 3, 5, 7, 11, 13, 17, 19]

while True:
    line = "quit"
    if line == "quit":
        break
def todo(): pass          # pass = do nothing placeholder
```

6.4 Pattern and classic loop problems

```
# Right triangle
for i in range(1, 4):
    print(" " * i)
# Fibonacci
a, b = 0, 1
for _ in range(8):
    print(a, end=" "); a, b = b, a + b      # 0 1 1 2 3 5 8 13
print()
# FizzBuzz
for i in range(1, 16):
    print("FizzBuzz" if i % 15 == 0 else "Fizz" if i % 3 == 0 else "Buzz" if i % 5 == 0 else i, end=" ")
print()
# Nested loop: common elements
A, B = [1, 2, 3], [2, 3, 4]
print([x for x in A if x in set(B)])        # [2, 3]
# Sum of digits / reverse a number
num = 1234
print(sum(int(d) for d in str(num)), int(str(num)[::-1])) # 10 4321
```

7. Functions and Argument Handling

7.1 Defining functions, defaults, return values

```
def greet(name, greeting="Hello"):          # default parameter
    """Return a greeting. (Docstring: appears in help(greet))"""
    return f"{greeting}, {name}!"

print(greet("Asha"))                        # Hello, Asha!
print(greet("Asha", greeting="Hi"))         # keyword argument
print(greet(greeting="Yo", name="Bob"))     # order does not matter with keywords

def stats(nums):                           # multiple return values = tuple
    return min(nums), max(nums), sum(nums) / len(nums)
lo, hi, avg = stats([2, 4, 9])
def no_return():                           # no return -> returns None
    pass
print(no_return())                         # None
```

7.2 The mutable default argument trap

```
def bad(item, bucket=[]):                  # the list is created ONCE at definition time
    bucket.append(item)
    return bucket
print(bad(1), bad(2))                     # [1, 2] [1, 2] <- shared state, surprise!

def good(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
print(good(1), good(2))                   # [1] [2]
```

7.3 *args and **kwargs

args** collects extra **positional** arguments into a **tuple**. *kwargs** collects extra **keyword** arguments into a **dict**. Order in a signature: positional, ***args**, keyword-only, ****kwargs**.

```
def total(*args):
    print(args, type(args))               # (1, 2, 3) <class 'tuple'>
    return sum(args)
print(total(1, 2, 3))                    # 6

def show(**kwargs):
    print(kwargs)                         # {'a': 1, 'b': 2}
    for k, v in kwargs.items():
        print(f"{k}={v}")
show(a=1, b=2)

def mixed(a, b=2, *args, key="k", **kwargs):
    return a, b, args, key, kwargs
print(mixed(1))                          # (1, 2, (), 'k', {})
print(mixed(1, 3, 4, 5, key="z", x=9))    # (1, 3, (4, 5), 'z', {'x': 9})

# Unpacking on the CALL side
nums = [1, 2, 3]
opts = {"sep": "-", "end": "!\n"}
print(*nums, **opts)                     # 1-2-3!
defaults = {"host": "localhost", "port": 5432}
overrides = {"port": 6543}
conn = {**defaults, **overrides}         # {'host': 'localhost', 'port': 6543}

# Forwarding arguments to another function (wrappers/decorators)
def wrapper(*args, **kwargs):
    return total(*args)
```

7.4 Positional-only and keyword-only parameters

```
def f(a, b, /, c, *, d):          # a,b positional-only ; c either ; d keyword-only
    return a + b + c + d
print(f(1, 2, 3, d=4))           # 10
print(f(1, 2, c=3, d=4))         # 10
# f(1, 2, 3, 4) -> TypeError (d must be keyword)
def g(*, retries=3, timeout=10):  # force readable calls: g(retries=5)
    return retries, timeout
```

7.5 Scope (LEGB), global, nonlocal

Name lookup order: **Local**, **Enclosing**, **Global**, **Built-in**.

```
count = 0
def inc_bad():
    # count += 1          # UnboundLocalError: assignment makes 'count' local
    pass
def inc():
    global count          # rebind the module-level name
    count += 1

def make_counter():
    n = 0
    def step():
        nonlocal n        # rebind the enclosing function's variable
        n += 1
        return n
    return step           # closure: step remembers n
c = make_counter()
print(c(), c(), c())     # 1 2 3
```

7.6 Lambda, map, filter, reduce

```
sq = lambda x: x * x          # anonymous one-expression function
print(sq(4))                  # 16
print(list(map(lambda x: x * 2, [1, 2, 3])))    # [2, 4, 6]
print(list(filter(lambda x: x % 2, [1, 2, 3, 4]))) # [1, 3]
from functools import reduce
print(reduce(lambda acc, x: acc + x, [1, 2, 3, 4])) # 10
print(reduce(lambda acc, x: acc * x, [1, 2, 3, 4], 1)) # 24 (with initial value)
print(sorted([("a", 2), ("b", 1)], key=lambda t: t[1])) # lambda as sort key
# Prefer comprehensions for readability: [x * 2 for x in nums]
from functools import partial
from_bin = partial(int, base=2)
print(from_bin("101"))        # 5 partial pre-fills arguments
```

7.7 Type hints and docstrings

```
from typing import Optional, Union, Any, Callable, Iterable

def fetch(ids: list[int], limit: Optional[int] = None) -> dict[int, str]:
    """Fetch names for ids.

    Args:
        ids: list of identifiers.
        limit: maximum rows, None for all.
    Returns:
        Mapping of id to name.
    """
    return {i: str(i) for i in ids[:limit]}

Number = Union[int, float]      # or int | float in 3.10+
def apply(fn: Callable[[int], int], xs: Iterable[int]) -> list[int]:
    return [fn(x) for x in xs]
# Hints are NOT enforced at runtime; they help IDEs, mypy and readers.
```

7.8 Recursion and memoization

```
def factorial(n):
    return 1 if n <= 1 else n * factorial(n - 1)
print(factorial(5))          # 120

from functools import lru_cache
@lru_cache(maxsize=None)    # memoization: cache results by arguments
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
print(fib(50))              # 12586269025 (instant; naive version is exponential)
print(fib.cache_info())

import sys
print(sys.getrecursionlimit())  # 1000 default: deep recursion -> RecursionError
```

8. Iterators, Generators, Decorators, Context Managers

8.1 Iterables vs iterators

An **iterable** has `__iter__` (list, str, dict). An **iterator** has `__next__` and remembers position; `iter(x)` gives one. `for` calls `iter()` then `next()` until `StopIteration`.

```
it = iter([10, 20])
print(next(it), next(it))    # 10 20
# next(it) -> StopIteration
print(next(iter([]), "default"))  # default (safe next)

class Countdown:
    def __init__(self, start): self.n = start
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        self.n -= 1
        return self.n + 1
print(list(Countdown(3)))    # [3, 2, 1]
```

8.2 Generators (memory-efficient, lazy)

```
def gen_squares(n):
    for i in range(n):
        yield i * i          # pauses here and resumes on next()

g = gen_squares(3)
print(next(g), next(g), next(g))  # 0 1 4
print(sum(gen_squares(1000)))    # never builds a 1000-item list

def read_chunks(items, size):    # batching pattern used in ETL / bulk inserts
    for i in range(0, len(items), size):
        yield items[i:i + size]
print(list(read_chunks([1, 2, 3, 4, 5], 2)))  # [[1, 2], [3, 4], [5]]

def evens():
    yield from (n for n in range(10) if n % 2 == 0)  # delegate to sub-iterator
# Generators are single-use: after exhaustion they yield nothing.
```

8.3 itertools highlights

```
import itertools as it
print(list(it.chain([1, 2], [3])))          # [1, 2, 3]
print(list(it.islice(it.count(10), 3)))    # [10, 11, 12]
print(list(it.permutations("abc", 2)[:3])) # [('a','b'), ('a','c'), ('b','a')]
print(list(it.combinations([1, 2, 3], 2))) # [(1,2), (1,3), (2,3)]
print(list(it.product("ab", [1, 2])))      # [('a',1), ('a',2), ('b',1), ('b',2)]
print(list(it.accumulate([1, 2, 3, 4])))   # [1, 3, 6, 10] running total
for key, grp in it.groupby(sorted(["ant", "art", "bee"]), key=lambda w: w[0]):
    print(key, list(grp))                  # a ['ant','art'] / b ['bee']
print(list(it.repeat("x", 3)), list(it.cycle("ab"))[:0])
print(list(it.zip_longest([1], [2, 3])))    # [(1, 2), (None, 3)]
# groupby only groups ADJACENT items, so sort first.
```

8.4 Decorators

A decorator is a function that takes a function and returns a wrapped function.

```
import functools, time

def timer(fn):
    @functools.wraps(fn)          # keep name/docstring of fn
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = fn(*args, **kwargs)
        print(f"{fn.__name__} took {time.perf_counter() - start:.4f}s")
        return result
    return wrapper

@timer
def slow(n):
    return sum(range(n))
slow(10_000)

# Decorator WITH arguments: one more layer
def retry(times=3, exceptions=(Exception,)):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return fn(*args, **kwargs)
                except exceptions as e:
                    print(f"attempt {attempt} failed: {e}")
                    if attempt == times:
                        raise
            return wrapper
        return decorator

@retry(times=2, exceptions=(ValueError,))
def flaky():
    raise ValueError("boom")
```

8.5 Context managers (with statement)

```
from contextlib import contextmanager, suppress, closing
import time

class Timer:
    # class based: __enter__ / __exit__
    def __enter__(self):
        self.t0 = time.perf_counter(); return self
    def __exit__(self, exc_type, exc, tb):
        print(f"elapsed {time.perf_counter() - self.t0:.3f}s")
        return False          # False: do not swallow exceptions

with Timer():
    sum(range(100_000))

@contextmanager
# generator based
```

```
def opened(path, mode="r"):
    f = open(path, mode)
    try:
        yield f
    finally:
        f.close()

with suppress(FileNotFoundError):
    open("missing.txt")
```

9. Exception Handling

9.1 try / except / else / finally

```
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("cannot divide by zero")
        return None
    except (TypeError, ValueError) as e:
        print("bad input:", e)
        return None
    except Exception as e:
        print("unexpected:", repr(e))
        raise
    else:
        print("no error happened")
        return result
    finally:
        print("cleanup always runs")

print(divide(10, 2))
print(divide(1, 0))
```

Order matters: Python checks **except** clauses top to bottom, so put specific exceptions before general ones. Never use a bare **except:** (it also catches **KeyboardInterrupt** and **SystemExit**). **finally** runs even when **return** or **break** is hit in **try**.

9.2 Common built-in exceptions

Exception	Typical cause	Example
ValueError	Right type, bad value	<code>int("abc")</code>
TypeError	Wrong type	<code>"a" + 1</code>
KeyError	Missing dict key	<code>{"x"}["x"]</code>
IndexError	List index out of range	<code>[1][5]</code>
AttributeError	Missing attribute/method	<code>None.upper()</code>
NameError	Undefined variable	<code>print(zzz)</code>
ZeroDivisionError	Division by zero	<code>1 / 0</code>
FileNotFoundError	Path does not exist	<code>open("nope.txt")</code>
PermissionError	No access rights	writing to protected path
ImportError / ModuleNotFoundError	Module missing	<code>import notamodule</code>
StopIteration	Iterator exhausted	<code>next(iter([]))</code>
RecursionError	Recursion too deep	endless recursion
OSError	Parent of file/network errors	disk full
json.JSONDecodeError	Invalid JSON (subclass of ValueError)	<code>json.loads("{}")</code>
requests.exceptions.RequestException	Any requests failure	timeout, DNS

Hierarchy: **BaseException** -> **Exception** -> (**ArithmeticError**, **LookupError** (**KeyError**, **IndexError**), **OSError** (**FileNotFoundError**...), **ValueError**, **TypeError**...). **KeyboardInterrupt** and **SystemExit** derive from **BaseException** directly.

9.3 raise, custom exceptions, chaining, assert

```
def set_age(age):
    if not isinstance(age, int):
        raise TypeError("age must be int")
    if age < 0:
        raise ValueError(f"age cannot be negative: {age}")
    return age

class PipelineError(Exception):
    """Base class for our application errors."""

class ValidationError(PipelineError):
    def __init__(self, field, message):
        super().__init__(f"{field}: {message}")
        self.field = field

try:
    raise ValidationError("email", "missing @")
except PipelineError as e:
    print(e, "| field =", e.field)
    # catches subclass via base class
    # email: missing @ | field = email

# Exception chaining: keep the root cause
def load(text):
    try:
        return int(text)
    except ValueError as e:
        raise PipelineError(f"cannot load {text!r}") from e
    # __cause__ set

# assert is for internal sanity checks (removed with python -O), not user input
assert 1 + 1 == 2, "math is broken"
```

9.4 Patterns used in real pipelines

```
import time, logging
log = logging.getLogger(__name__)

# 1. Retry with exponential backoff
def with_retry(fn, attempts=4, base_delay=1.0, retry_on=(ConnectionError, TimeoutError)):
    for i in range(attempts):
        try:
            return fn()
        except retry_on as e:
            if i == attempts - 1:
                raise
            delay = base_delay * (2 ** i)
            log.warning("attempt %d failed (%s); retrying in %.1fs", i + 1, e, delay)
            time.sleep(delay)

# 2. Bad-record quarantine: do not fail the whole batch for one bad row
def parse_rows(rows):
    good, bad = [], []
    for i, row in enumerate(rows):
        try:
            good.append({"id": int(row["id"]), "amt": float(row["amt"])})
        except (KeyError, ValueError) as e:
            bad.append({"line": i, "row": row, "error": str(e)})
    return good, bad
good, bad = parse_rows([{"id": "1", "amt": "9.5"}, {"id": "x", "amt": "1"}, {"amt": "2"}])
print(len(good), len(bad))
# 1 2

# 3. EAFP ("easier to ask forgiveness") vs LBYL ("look before you leap")
d = {"a": 1}
try:
    v = d["b"]
except KeyError:
    v = 0
v = d["b"] if "b" in d else 0
v = d.get("b", 0)

# 4. Multiple errors (3.11+): ExceptionGroup and except*
# try: raise ExceptionGroup("batch", [ValueError("a"), TypeError("b")])
```

```
# except* ValueError as eg: ...
```

Best practices: catch the narrowest exception, keep the `try` body small, never silently `pass`, log with `log.exception(...)` (includes traceback), clean up resources with `with` or `finally`, and raise exceptions with helpful messages.

10. File Handling: Read, Write, Edit

10.1 `open()` modes

Mode	Meaning	Behaviour
<code>r</code>	read (default)	error if file missing
<code>w</code>	write	creates, truncates existing content
<code>a</code>	append	creates, writes at end
<code>x</code>	exclusive create	error if file exists
<code>r+</code>	read and write	file must exist, no truncate
<code>w+ / a+</code>	write/append plus read	
<code>b</code> suffix	binary (<code>rb</code> , <code>wb</code>)	bytes instead of str
<code>t</code> suffix	text (default)	decoded with encoding

Always use `with open(...)` (auto-closes even on error) and pass `encoding="utf-8"` for text, plus `newline=""` when using the `csv` module.

10.2 Reading and writing text

```
# WRITE (creates/overwrites)
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("line one\n")
    f.write("line two\n")
    f.writelines(["line three\n", "line four\n"]) # no newline added automatically
    print("line five", file=f)                  # print can target a file

# APPEND
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("appended\n")

# READ variants
with open("notes.txt", encoding="utf-8") as f:
    whole = f.read() # entire file as ONE string (small files only)
with open("notes.txt", encoding="utf-8") as f:
    first = f.readline() # one line including trailing "\n"
    rest = f.readlines() # list of remaining lines
with open("notes.txt", encoding="utf-8") as f:
    for line in f: # BEST for big files: lazy, one line at a time
        print(line.rstrip("\n"))
with open("notes.txt", encoding="utf-8") as f:
    print(f.read(5)) # read 5 characters
    print(f.tell()) # current position: 5
    f.seek(0) # jump back to start
    lines = [ln.strip() for ln in f if ln.strip()] # skip blank lines

# Binary copy in chunks
with open("in.bin", "rb") as src, open("out.bin", "wb") as dst:
    while chunk := src.read(1024 * 1024):
        dst.write(chunk)
```

10.3 Editing a file (the edit process)

Text files cannot be edited "in the middle" in place; the safe pattern is **read -> modify in memory (or stream) -> write back**. For large files stream to a temp file and atomically replace the original.

```
from pathlib import Path
import os, tempfile

# A) Small file: read, modify, write back
p = Path("config.txt")
p.write_text("host=localhost\nport=5432\ndebug=true\n", encoding="utf-8")
lines = p.read_text(encoding="utf-8").splitlines()
lines = [ln.replace("debug=true", "debug=false") for ln in lines] # replace
lines.insert(1, "user=admin") # insert at line 2
lines = [ln for ln in lines if not ln.startswith("port=")] # delete matching
lines.append("# edited") # append
p.write_text("\n".join(lines) + "\n", encoding="utf-8")

# B) Large file: stream line by line into a temp file, then atomically swap
def replace_in_file(path, old, new):
    path = Path(path)
    fd, tmp_name = tempfile.mkstemp(dir=path.parent) # same folder = same filesystem
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as tmp, \
            open(path, encoding="utf-8") as src:
            for line in src:
                tmp.write(line.replace(old, new))
        os.replace(tmp_name, path) # atomic on same filesystem
    except Exception:
        os.unlink(tmp_name) # clean up on failure
        raise

# C) Update a specific line number
def update_line(path, lineno, text): # lineno is 1-based
    with open(path, "r+", encoding="utf-8") as f:
        lines = f.readlines()
        lines[lineno - 1] = text + "\n"
        f.seek(0); f.writelines(lines); f.truncate() # truncate in case it got shorter

# D) Stdlib in-place editing (creates a .bak backup automatically)
import fileinput
for line in fileinput.input("config.txt", inplace=True, backup=".bak"):
    print(line.replace("localhost", "127.0.0.1"), end="") # print goes INTO the file

# E) Overwrite header of CSV, add a line to the top
def prepend(path, text):
    old = Path(path).read_text(encoding="utf-8")
    Path(path).write_text(text + "\n" + old, encoding="utf-8")
```

10.4 CSV

```

import csv

# Write
rows = [{"id": 1, "name": "Asha", "amt": 10.5}, {"id": 2, "name": "Bob", "amt": 7}]
with open("out.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["id", "name", "amt"])
    w.writeheader()
    w.writerows(rows)

# Read as dicts (header row becomes keys; every value is a str!)
with open("out.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["name"], float(row["amt"]))

# Read as lists / other delimiter / quoting
with open("data.tsv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f, delimiter="\t", quotechar="'")
    header = next(reader)
    data = list(reader)
with open("plain.csv", "w", newline="") as f:
    csv.writer(f).writerow(["a", "b,with,comma"]) # quotes fields automatically

```

10.5 JSON, JSON Lines, pickle

```

import json
obj = {"id": 1, "tags": ["a", "b"], "ok": True, "none": None, "price": 9.5}
s = json.dumps(obj, indent=2, sort_keys=True) # dict -> str
back = json.loads(s) # str -> dict (True -> true, None -> null)

with open("data.json", "w", encoding="utf-8") as f:
    json.dump(obj, f, indent=2, ensure_ascii=False)
with open("data.json", encoding="utf-8") as f:
    data = json.load(f) # file -> Python object

# Not serialisable by default: datetime, Decimal, set -> supply default=
from datetime import datetime
print(json.dumps({"t": datetime(2026, 9, 24)}, default=str)) # {"t": "2026-09-24 00:00:00"}

# JSON Lines / NDJSON: one JSON object per line, ideal for big data, appendable
with open("events.jsonl", "w", encoding="utf-8") as f:
    for rec in rows:
        f.write(json.dumps(rec) + "\n")
with open("events.jsonl", encoding="utf-8") as f:
    records = [json.loads(line) for line in f if line.strip()]

import pickle # Python-only binary format
with open("obj.pkl", "wb") as f:
    pickle.dump(obj, f)
# NEVER unpickle data from untrusted sources (can execute code).

```

10.6 pathlib, shutil, temp files, compression

```

from pathlib import Path
import shutil, tempfile, gzip

base = Path("data") / "raw" / "2026" # / operator joins paths
base.mkdir(parents=True, exist_ok=True) # like mkdir -p
f = base / "a.txt"
f.write_text("hello", encoding="utf-8")
print(f.exists(), f.is_file(), f.suffix, f.stem, f.name, f.parent) # True True .txt a a.txt data/raw/2026
print(f.stat().st_size) # size in bytes
print(list(base.glob("*.txt")), list(Path("data").rglob("*.txt"))) # rglob = recursive
f.rename(base / "b.txt") # move/rename ; f.unlink() deletes
print(Path.cwd(), Path.home(), f.resolve()) # absolute path

shutil.copy("src.txt", "dst.txt") # copy file (copy2 keeps metadata)
shutil.copytree("dir_a", "dir_b") # copy directory tree
shutil.move("a.txt", "archive/a.txt")
shutil.rmtree("old_dir", ignore_errors=True) # delete directory tree (careful!)

```

```

with tempfile.TemporaryDirectory() as tmp:      # auto-deleted folder
    Path(tmp, "x.txt").write_text("temp")

with gzip.open("big.txt.gz", "wt", encoding="utf-8") as g:  # transparent compression
    g.write("compressed text\n")
with gzip.open("big.txt.gz", "rt", encoding="utf-8") as g:
    print(g.read())
shutil.make_archive("backup", "zip", "data")      # zip a folder -> backup.zip

```

10.7 Big-file strategies

```

# 1) Stream lines, aggregate on the fly (O(1) memory)
error_count = 0
with open("server.log", encoding="utf-8", errors="replace") as f:
    for line in f:
        if " ERROR " in line:
            error_count += 1

# 2) Process in batches for bulk load
def batches(iterable, n):
    batch = []
    for item in iterable:
        batch.append(item)
        if len(batch) == n:
            yield batch; batch = []
    if batch:
        yield batch

# 3) pandas in chunks
import pandas as pd
total = 0
for chunk in pd.read_csv("huge.csv", chunksize=100_000):
    total += chunk["amount"].sum()

# 4) Word count of a file
from collections import Counter
with open("book.txt", encoding="utf-8") as f:
    wc = Counter(w.lower().strip(".,!?") for line in f for w in line.split())
print(wc.most_common(5))

```

11. os, sys, and Related Standard Library

11.1 os module

```

import os

print(os.getcwd())          # current working directory
os.chdir("/tmp")            # change directory
print(os.listdir("."))      # names in a directory
os.mkdir("one"); os.makedirs("a/b/c", exist_ok=True)
os.rename("old.txt", "new.txt"); os.remove("file.txt")  # delete file
os.rmdir("empty_dir")       # delete EMPTY directory
print(os.path.join("data", "in", "f.csv"))  # portable path building
print(os.path.exists("x"), os.path.isfile("x"), os.path.isdir("x"))
print(os.path.basename("/a/b/f.txt"), os.path.dirname("/a/b/f.txt"))  # f.txt /a/b
print(os.path.splitext("report.final.csv"))  # ('report.final', '.csv')
print(os.path.abspath("."), os.path.getsize("f.txt"))
print(os.cpu_count(), os.getpid(), os.name, os.sep)

# Walk a directory tree
for root, dirs, files in os.walk("data"):
    for name in files:
        if name.endswith(".csv"):
            print(os.path.join(root, name))

# Environment variables (config and secrets belong here, never in code)
print(os.environ.get("HOME"))  # None if missing
db_user = os.getenv("DB_USER", "default_user")

```

```
os.environ["MY_FLAG"] = "1"           # visible to child processes
password = os.environ["DB_PASSWORD"]  # KeyError if not set -> fail fast
```

11.2 sys module

```
import sys

print(sys.argv)           # ['script.py', 'arg1', 'arg2'] (argv[0] is script name)
print(sys.version)        # interpreter version string
print(sys.version_info >= (3, 10)) # numeric version compare
print(sys.platform)       # 'linux', 'win32', 'darwin'
print(sys.path[:2])       # module search path (list, can be modified)
print(sys.executable)     # path of the running python
print(sys.modules.keys() is not None) # loaded modules
print(sys.getsizeof([1, 2, 3])) # shallow object size in bytes
print(sys.maxsize)        # largest list index / Py_ssize_t

sys.stdout.write("no newline added\n") # low level print
print("to stderr", file=sys.stderr)    # errors and logs go to stderr
line = sys.stdin.readline()            # read piped input: cat f | python s.py
for line in sys.stdin:                 # loop over piped lines
    pass

if len(sys.argv) < 2:
    print("usage: script.py <file>", file=sys.stderr)
    sys.exit(2)                       # non-zero exit code = failure
sys.exit(0)                          # success (raises SystemExit)
```

Exit codes matter for schedulers (Airflow, cron, CI): 0 = success, anything else = failure.

11.3 subprocess, glob, datetime, time

```
import subprocess, glob, time
from datetime import datetime, timedelta, timezone, date

res = subprocess.run(["ls", "-l"], capture_output=True, text=True, check=False)
print(res.returncode, res.stdout[:50], res.stderr)
# check=True raises CalledProcessError on non-zero exit; list args avoid shell injection
# NEVER pass untrusted input with shell=True

print(glob.glob("data/*.csv"), glob.glob("**/*.py", recursive=True))

now = datetime.now()           # naive local time
utc = datetime.now(timezone.utc) # timezone-aware UTC (prefer for pipelines)
print(now.isoformat(), utc.strftime("%Y-%m-%dT%H:%M:%SZ"))
tomorrow = now + timedelta(days=1)
delta = datetime(2026, 12, 25) - datetime(2026, 9, 24)
print(delta.days, date.today(), date.today().weekday()) # weekday(): Monday=0
print(datetime.fromisoformat("2026-09-24T10:00:00"))
t0 = time.perf_counter(); time.sleep(0.01); print(time.perf_counter() - t0) # timing
print(int(time.time()))           # epoch seconds
```

12. Command-Line Arguments: sys.argv and argparse

Interview note: "pyargs" is usually shorthand for Python argument parsing, i.e. the standard-library **argparse** module (sometimes **getopt**, or third-party **click** / **typer**). All are covered here.

12.1 Raw sys.argv

```
# run: python load.py data.csv 100
import sys
def main():
    if len(sys.argv) != 3:
        print("usage: load.py <file> <batch_size>", file=sys.stderr)
        return 2
    path, batch = sys.argv[1], int(sys.argv[2])    # everything arrives as str
    print(path, batch)
    return 0
if __name__ == "__main__":
    sys.exit(main())
```

12.2 argparse

```
import argparse

def build_parser():
    p = argparse.ArgumentParser(
        prog="etl", description="Load a file into a table.",
        formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    p.add_argument("input", help="input file path")           # positional, required
    p.add_argument("-t", "--table", required=True, help="target table") # required option
    p.add_argument("-b", "--batch-size", type=int, default=1000) # typed + default
    p.add_argument("--mode", choices=["append", "overwrite"], default="append")
    p.add_argument("-v", "--verbose", action="store_true")    # flag: True if present
    p.add_argument("-q", "--quiet", action="store_false", dest="loud") # flag defaults True
    p.add_argument("--tag", action="append", default=[])      # repeatable: --tag a --tag b
    p.add_argument("--cols", nargs="+", metavar="COL")        # one or more values
    p.add_argument("--limit", nargs="?", const=10, type=int)   # optional value
    p.add_argument("--version", action="version", version="etl 1.0")
    return p

def main(argv=None):
    args = build_parser().parse_args(argv)    # argv=None reads sys.argv[1:]
    print(args.input, args.table, args.batch_size, args.mode, args.verbose, args.tag)
    print(vars(args))                        # namespace -> dict

main(["a.csv", "-t", "orders", "-b", "500", "--tag", "x", "--tag", "y", "-v"])
# python etl.py a.csv -t orders -b 500 --mode overwrite
# python etl.py -h          -> auto-generated help
```

Argparse exits with code 2 and usage text on invalid arguments. Passing **argv** to **main()** makes it unit-testable.

```
# Sub-commands (like git add / git commit)
import argparse
p = argparse.ArgumentParser(prog="tool")
sub = p.add_subparsers(dest="cmd", required=True)
ld = sub.add_parser("load"); ld.add_argument("file")
ex = sub.add_parser("export"); ex.add_argument("--fmt", default="csv")
args = p.parse_args(["load", "a.csv"])
if args.cmd == "load":
    print("loading", args.file)

# Mutually exclusive options and custom validation type
g = argparse.ArgumentParser().add_mutually_exclusive_group()
def positive_int(s):
    n = int(s)
    if n <= 0:
        raise argparse.ArgumentTypeError("must be > 0")
    return n
```

```
# p.add_argument("--n", type=positive_int)

# Combine CLI args with environment variables (typical config precedence: CLI > env > default)
import os
p2 = argparse.ArgumentParser()
p2.add_argument("--host", default=os.getenv("DB_HOST", "localhost"))
```

13. Logging

13.1 Why logging instead of print

Levels, timestamps, module names, multiple destinations (console, files, remote), and one switch to change verbosity. Level order: `DEBUG(10) < INFO(20) < WARNING(30) < ERROR(40) < CRITICAL(50)`; default threshold is `WARNING`.

13.2 Quick setup and usage

```
import logging

logging.basicConfig(
    level=logging.INFO,                                # threshold
    format="%(asctime)s | %(levelname)-8s | %(name)s: %(lineno)d | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    handlers=[logging.StreamHandler(), logging.FileHandler("etl.log", encoding="utf-8")],
)
log = logging.getLogger(__name__)    # one logger per module, named after the module

log.debug("hidden at INFO level")
log.info("loaded %d rows from %s", 1200, "orders.csv")    # lazy % formatting (preferred)
log.warning("null ratio high: %.1f%%", 12.5)
log.error("failed to connect to %s", "db1")
log.critical("out of disk space")

try:
    1 / 0
except ZeroDivisionError:
    log.exception("calculation failed")    # ERROR + full traceback (use inside except)
log.info("user=%s", "bob", extra={"run_id": "abc"})    # extra fields for formatters
```

Format codes: `%(asctime)s` time, `%(levelname)s` level, `%(name)s` logger name, `%(message)s` text, `%(filename)s`, `%(funcName)s`, `%(lineno)d`, `%(process)d`, `%(thread)d`. `basicConfig` only works the first time (unless `force=True`).

13.3 Handlers, formatters, rotation, dictConfig

```
import logging, logging.handlers, sys

def setup_logging(level="INFO", logfile="app.log"):
    root = logging.getLogger()
    root.setLevel(level)
    fmt = logging.Formatter("%(asctime)s %(levelname)s %(name)s - %(message)s")

    console = logging.StreamHandler(sys.stdout)
    console.setLevel(logging.INFO); console.setFormatter(fmt)

    rotating = logging.handlers.RotatingFileHandler(    # size based rotation
        logfile, maxBytes=5_000_000, backupCount=3, encoding="utf-8")
    rotating.setLevel(logging.DEBUG); rotating.setFormatter(fmt)

    daily = logging.handlers.TimedRotatingFileHandler(    # time based rotation
        "daily.log", when="midnight", backupCount=7)
    root.handlers.clear()
    root.addHandler(console); root.addHandler(rotating)

setup_logging()
logging.getLogger("urllib3").setLevel(logging.WARNING)    # silence noisy libraries
child = logging.getLogger("etl.extract")    # hierarchy: propagates to "etl"
```

```
# Config as a dictionary (same idea used in Django, or loaded from YAML/JSON)
from logging.config import dictConfig
dictConfig({
    "version": 1,
    "formatters": {"std": {"format": "%(asctime)s %(levelname)s %(name)s %(message)s"}},
    "handlers": {"console": {"class": "logging.StreamHandler", "formatter": "std"}},
    "root": {"level": "INFO", "handlers": ["console"]},
})

# Structured (JSON) logs: easy to search in Splunk / ELK / Datadog
import json
class JsonFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({"ts": self.formatTime(record), "level": record.levelname,
                          "logger": record.name, "msg": record.getMessage()})
```

Best practices: `getLogger(__name__)` in every module; configure handlers only once at the entry point (never in library code); use `%s` placeholders not f-strings; never log secrets or full PII; log row counts, durations and IDs (`run_id`) in pipelines; use `log.exception` in `except` blocks.

14. Object-Oriented Programming (OOP)

14.1 Classes, objects, `__init__`, instance vs class attributes

A **class** is a blueprint; an **object** (instance) is a concrete thing built from it. `self` is the instance passed automatically as the first argument of instance methods.

```
class Employee:
    company = "Acme"                # class attribute: shared by ALL instances
    count = 0

    def __init__(self, name, salary): # initializer (constructor-like)
        self.name = name             # instance attributes: per object
        self.salary = salary
        Employee.count += 1          # modify class attribute via class name

    def raise_salary(self, pct):      # instance method
        self.salary *= 1 + pct / 100
        return self

    def __repr__(self):              # unambiguous, for developers/debugging
        return f"Employee(name={self.name!r}, salary={self.salary})"

    def __str__(self):               # readable, used by print()/str()
        return f"{self.name} earns {self.salary:.0f}"

e1, e2 = Employee("Asha", 1000), Employee("Bob", 900)
print(e1)                          # Asha earns 1000
print(repr(e2))                     # Employee(name='Bob', salary=900)
print(Employee.count, e1.company)   # 2 Acme
e1.company = "Zeta"                 # creates an INSTANCE attribute that shadows class one
print(e1.company, e2.company)       # Zeta Acme
print(e1.raise_salary(10).raise_salary(10).salary) # chaining works because we return self
print(hasattr(e1, "name"), getattr(e1, "age", None), e1.__dict__)
```

14.2 Instance, class, and static methods

```

class Date:
    def __init__(self, y, m, d):
        self.y, self.m, self.d = y, m, d

    def show(self):
        # instance method: gets self
        return f"{self.y}-{self.m:02d}-{self.d:02d}"

    @classmethod
    def from_string(cls, s):
        # gets the CLASS: alternative constructor / factory
        y, m, d = map(int, s.split("-"))
        return cls(y, m, d)
        # cls works correctly with subclasses too

    @staticmethod
    def is_leap(year):
        # gets nothing: utility grouped with class
        return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)

d = Date.from_string("2026-09-24")
print(d.show(), Date.is_leap(2028))    # 2026-09-24 True

```

Rule of thumb: need instance data -> instance method; need the class (factories, class-level state) -> `@classmethod`; need neither -> `@staticmethod` (or a module-level function).

14.3 Encapsulation: public, `_protected`, `__private`, properties

Python has no true private members; it uses naming conventions.

```

class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._bank = "XYZ"
        self.__balance = balance
        # public
        # protected by convention: "internal use"
        # name-mangled to _Account__balance

    @property
    def balance(self):
        # getter: acts like an attribute
        return self.__balance

    @balance.setter
    def balance(self, value):
        # setter with validation
        if value < 0:
            raise ValueError("balance cannot be negative")
        self.__balance = value

    @property
    def summary(self):
        # read-only computed property (no setter)
        return f"{self.owner}: {self.__balance}"

    def deposit(self, amt):
        if amt <= 0:
            raise ValueError("deposit must be positive")
        self.__balance += amt

    def withdraw(self, amt):
        if amt > self.__balance:
            raise ValueError("insufficient funds")
        self.__balance -= amt

a = Account("Asha", 100)
a.deposit(50); a.withdraw(30)
print(a.balance)
a.balance = 200
# print(a.__balance)
print(a._Account__balance)
# a.summary = "x"
# 120
# goes through the setter
# AttributeError (mangled)
# 200 (still reachable: privacy is by convention)
# AttributeError: no setter

```

14.4 Inheritance, super(), method overriding

```
class Person:
    def __init__(self, name):
        self.name = name
    def role(self):
        return "person"
    def intro(self):
        return f"I am {self.name}, a {self.role()}"

class Manager(Person):
    # single inheritance
    def __init__(self, name, team):
        super().__init__(name)
        self.team = team
    def role(self):
        # overriding
        return "manager"
    def intro(self):
        return super().intro() + f" of {len(self.team)} people" # extend parent behaviour

m = Manager("Asha", ["a", "b"])
print(m.intro())
# I am Asha, a manager of 2 people
print(isinstance(m, Person), isinstance(m, Manager), issubclass(Manager, Person)) # True True True
print(type(m) is Person)
# False (isinstance respects inheritance, type() is exact)
```

14.5 Multiple inheritance and MRO

```
class A:
    def hello(self): return "A"
class B(A):
    def hello(self): return "B>" + super().hello()
class C(A):
    def hello(self): return "C>" + super().hello()
class D(B, C):
    def hello(self): return "D>" + super().hello()

print(D().hello())
# D>B>C>A super() follows the MRO, not just the parent
print([k.__name__ for k in D.__mro__]) # ['D', 'B', 'C', 'A', 'object']
# MRO = C3 linearisation: children before parents, left-to-right order preserved.

# Mixins: small classes that add one capability, used with multiple inheritance
class JsonMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)
class User(JsonMixin):
    def __init__(self, name): self.name = name
print(User("Bob").to_json())
# {"name": "Bob"}
```

14.6 Polymorphism, duck typing, abstract base classes

```
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    # cannot be instantiated
    @abstractmethod
    def area(self): ...
    @abstractmethod
    def perimeter(self): ...
    def describe(self):
        # concrete method shared by subclasses
        return f"{type(self).__name__}: area={self.area():.2f}"

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return math.pi * self.r ** 2
    def perimeter(self): return 2 * math.pi * self.r

class Rect(Shape):
    def __init__(self, w, h): self.w, self.h = w, h
    def area(self): return self.w * self.h
    def perimeter(self): return 2 * (self.w + self.h)
```

```
# Polymorphism: same call, behaviour depends on the object
for s in [Circle(1), Rect(2, 3)]:
    print(s.describe())           # Circle: area=3.14 / Rect: area=6.00
# Shape() -> TypeError: Can't instantiate abstract class
# A subclass that forgets an abstract method also cannot be instantiated.

# Duck typing: "if it walks like a duck..." no common base needed
class Duck:
    def speak(self): return "Quack"
class Dog:
    def speak(self): return "Woof"
for animal in (Duck(), Dog()):
    print(animal.speak())
```

14.7 Special (dunder) methods

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        return f"Vector({self.x}, {self.y})"
    def __add__(self, o):
        return Vector(self.x + o.x, self.y + o.y)           # v1 + v2
    def __sub__(self, o):
        return Vector(self.x - o.x, self.y - o.y)
    def __mul__(self, k):
        return Vector(self.x * k, self.y * k)               # v * 3
    def __rmul__(self, k):
        return self * k                                     # 3 * v
    def __neg__(self):
        return Vector(-self.x, -self.y)
    def __eq__(self, o):
        return isinstance(o, Vector) and (self.x, self.y) == (o.x, o.y)
    def __lt__(self, o):
        return abs(self) < abs(o)                           # enables sorting
    def __abs__(self):
        return (self.x ** 2 + self.y ** 2) ** 0.5
    def __bool__(self):
        return bool(self.x or self.y)
    def __hash__(self):
        return hash((self.x, self.y))                       # define with __eq__ for set/dict use
    def __len__(self):
        return 2
    def __getitem__(self, i):
        return (self.x, self.y)[i]                          # indexing + iteration
    def __iter__(self):
        yield self.x; yield self.y
    def __contains__(self, v):
        return v in (self.x, self.y)
    def __call__(self, k):
        return self * k                                     # instance behaves like a function

v, w = Vector(1, 2), Vector(3, 4)
print(v + w, v * 3, 3 * v, -v)                             # Vector(4, 6) Vector(3, 6) Vector(3, 6) Vector(-1, -2)
print(v == Vector(1, 2), sorted([w, v]), abs(w))           # True [Vector(1, 2), Vector(3, 4)] 5.0
x, y = v
print(len(v), v[0], 2 in v, v(10), {v: "a"}[Vector(1, 2)])
```

Dunder	Triggered by	Dunder	Triggered by
<code>__init__</code>	<code>Class(...)</code> after creation	<code>__new__</code>	actual object creation
<code>__repr__</code> / <code>__str__</code>	<code>repr()</code> / <code>print()</code>	<code>__del__</code>	garbage collection (avoid)
<code>__eq__</code> , <code>__lt__</code> ...	<code>==</code> , <code><</code> ...	<code>__hash__</code>	<code>hash()</code> , set/dict keys
<code>__len__</code>	<code>len(x)</code>	<code>__getitem__</code>	<code>x[i]</code>
<code>__iter__</code> / <code>__next__</code>	for loops	<code>__contains__</code>	<code>in</code>
<code>__enter__</code> / <code>__exit__</code>	<code>with</code>	<code>__call__</code>	<code>x()</code>
<code>__getattr__</code>	missing attribute lookup	<code>__setattr__</code>	<code>x.a = v</code>

14.8 dataclasses, enums, __slots__

```
from dataclasses import dataclass, field, asdict, astuple
from enum import Enum, auto

@dataclass(order=True, frozen=False)           # auto __init__, __repr__, __eq__ (+ ordering)
class Product:
    name: str
    price: float = 0.0
    tags: list = field(default_factory=list)     # mutable default must use field()
    def total(self, qty): return self.price * qty

p = Product("pen", 1.5, ["office"])
print(p)                                       # Product(name='pen', price=1.5, tags=['office'])
```

```

print(asdict(p), p == Product("pen", 1.5, ["office"]))

@dataclass(frozen=True)                # immutable and hashable
class Point:
    x: int
    y: int
# Point(1, 2).x = 5 -> FrozenInstanceError

@dataclass
class Order:
    id: int
    qty: int
    def __post_init__(self):            # validation after generated __init__
        if self.qty <= 0:
            raise ValueError("qty must be positive")

class Status(Enum):
    NEW = auto()
    DONE = auto()
class Color(Enum):
    RED = "r"
    GREEN = "g"
print(Color("r"), Color.RED.name, Color.RED.value, list(Color))  # Color.RED RED r [...]

class Slim:
    __slots__ = ("a", "b")              # no per-instance __dict__: saves memory, blocks new attrs
    def __init__(self, a, b): self.a, self.b = a, b

```

14.9 Composition, design patterns, class-based tools

```

# Composition: "has-a" (often better than deep inheritance)
class Engine:
    def start(self): return "engine on"
class Car:
    def __init__(self): self.engine = Engine()
    def start(self): return self.engine.start()

# Singleton via module-level instance or __new__
class Config:
    _inst = None
    def __new__(cls, *a, **k):
        if cls._inst is None:
            cls._inst = super().__new__(cls)
        return cls._inst
print(Config() is Config())            # True

# Factory
class CsvReader:
    def read(self, p): return f"csv:{p}"
class JsonReader:
    def read(self, p): return f"json:{p}"
def reader_for(path):
    return {"csv": CsvReader, "json": JsonReader}[path.rsplit(".", 1)[-1]]()
print(reader_for("a.csv").read("a.csv")) # csv:a.csv

# Strategy via first-class functions
def clean_upper(s): return s.strip().upper()
def clean_lower(s): return s.strip().lower()
class Cleaner:
    def __init__(self, strategy): self.strategy = strategy
    def run(self, rows): return [self.strategy(r) for r in rows]
print(Cleaner(clean_upper).run([" a ", "b "])) # ['A', 'B']

# Class-based decorator with state
class CountCalls:
    def __init__(self, fn): self.fn, self.calls = fn, 0
    def __call__(self, *a, **k):
        self.calls += 1
        return self.fn(*a, **k)
@CountCalls
def ping(): return "pong"
ping(); ping(); print(ping.calls)      # 2

```

```
# Protocol (structural typing, 3.8+)
from typing import Protocol
class Writer(Protocol):
    def write(self, s: str) -> int: ...
def dump(w: Writer, s: str): w.write(s)
```

14.10 Full worked example: inventory system

```
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
import logging
log = logging.getLogger(__name__)

class OutOfStock(Exception):
    pass

@dataclass
class Item:
    sku: str
    name: str
    price: float
    qty: int = 0

class Repository(ABC):
    @abstractmethod
    def get(self, sku): ...
    @abstractmethod
    def save(self, item): ...

class MemoryRepo(Repository):
    def __init__(self): self._data = {}
    def get(self, sku):
        try:
            return self._data[sku]
        except KeyError:
            raise KeyError(f"unknown sku {sku}") from None
    def save(self, item): self._data[item.sku] = item
    def __iter__(self): return iter(self._data.values())
    def __len__(self): return len(self._data)

class Inventory:
    def __init__(self, repo: Repository): # dependency injection: easy to test/swap
        self.repo = repo
    def add_stock(self, sku, name, price, qty):
        try:
            item = self.repo.get(sku)
            item.qty += qty
        except KeyError:
            item = Item(sku, name, price, qty)
            self.repo.save(item)
    def remove_stock(self, sku, qty):
        item = self.repo.get(sku)
        if item.qty < qty:
            raise OutOfStock(f"{sku}: have {item.qty}, need {qty}")
        item.qty -= qty
        log.info("removed %d of %s", qty, sku)
    def total_value(self):
        return sum(i.price * i.qty for i in self.repo)

inv = Inventory(MemoryRepo())
inv.add_stock("A1", "pen", 2.0, 10)
inv.add_stock("A1", "pen", 2.0, 5)
inv.remove_stock("A1", 3)
print(inv.total_value()) # 24.0
try:
    inv.remove_stock("A1", 100)
except OutOfStock as e:
    print("error:", e) # error: A1: have 12, need 100
```

14.11 OOP interview cheat sheet

Concept	One-line answer
Four pillars	Encapsulation, Abstraction, Inheritance, Polymorphism
<code>__init__</code> vs <code>__new__</code>	<code>__new__</code> creates the object, <code>__init__</code> initialises it
<code>@classmethod</code> vs <code>@staticmethod</code>	gets <code>cls</code> vs gets nothing
<code>__str__</code> vs <code>__repr__</code>	user-friendly vs developer/debug (fallback for <code>__str__</code>)
Class vs instance attribute	shared across instances vs per object; assignment via instance shadows
Shallow vs deep copy	shared nested objects vs fully independent
<code>is</code> vs <code>==</code>	identity vs equality
MRO	order of base class lookup (C3), see <code>Class.__mro__</code>
<code>super()</code>	next class in MRO, not necessarily the direct parent
Abstract class	<code>ABC</code> + <code>@abstractmethod</code> , cannot be instantiated
Property	method exposed like an attribute, enables validation
Duck typing	behaviour matters, not the declared type
Composition vs inheritance	has-a vs is-a; prefer composition for flexibility
Overloading	no method overloading in Python: use defaults, <code>*args</code> , or <code>functools.singledispatch</code>
<code>__slots__</code>	fixed attributes, less memory
Name mangling	<code>__x</code> -> <code>_Class__x</code> to avoid subclass clashes
Metaclass	class of a class (<code>type</code>), used by frameworks like ORMs; rarely needed
Immutable object	<code>frozen</code> dataclass, tuple, namedtuple

15. Working with APIs (REST) and Pagination

15.1 HTTP and REST basics

Item	Meaning
GET	read data (safe, idempotent)
POST	create / submit data (not idempotent)
PUT / PATCH	replace / partially update (PUT is idempotent)
DELETE	remove
2xx	success: 200 OK, 201 Created, 204 No Content
3xx	redirect: 301, 302, 304
4xx	client error: 400 bad request, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict, 422 validation, 429 too many requests
5xx	server error: 500, 502, 503, 504 (retry these; do NOT blindly retry 4xx except 429)
Headers	<code>Authorization</code> , <code>Content-Type</code> , <code>Accept</code> , <code>Retry-After</code> , <code>Link</code> , <code>User-Agent</code>
Query string	<code>?page=2&limit=100</code> filters / paging
Body	JSON payload for POST/PUT/PATCH

15.2 requests library: GET, POST, params, headers

```
import requests

# GET with query params -> https://api.example.com/v1/users?active=true&limit=50
r = requests.get(
    "https://api.example.com/v1/users",
    params={"active": "true", "limit": 50},          # encoded for you
    headers={"Accept": "application/json"},
    timeout=(3.05, 30),                             # (connect, read) seconds: ALWAYS set
)
```

```

print(r.status_code, r.ok, r.url)                # 200 True final-url
print(r.headers.get("Content-Type"))
data = r.json()                                # parse JSON body -> dict/list
text = r.text                                  # body as str ; r.content = bytes
r.raise_for_status()                           # raises HTTPError for 4xx/5xx

# POST JSON body (json= sets Content-Type automatically)
r = requests.post("https://api.example.com/v1/users",
                  json={"name": "Asha", "role": "DE"}, timeout=30)
# POST form data: requests.post(url, data={"a": 1}); file upload: files={"f": open("x.csv", "rb")}
requests.put(url := "https://api.example.com/v1/users/1", json={"name": "B"}, timeout=30)
requests.delete(url, timeout=30)

# Error handling around a call
try:
    r = requests.get("https://api.example.com/v1/x", timeout=10)
    r.raise_for_status()
    payload = r.json()
except requests.exceptions.Timeout:
    print("timed out")
except requests.exceptions.HTTPError as e:
    print("HTTP error", e.response.status_code, e.response.text[:200])
except requests.exceptions.ConnectionError:
    print("network problem")
except requests.exceptions.JSONDecodeError:
    print("response was not JSON")
except requests.exceptions.RequestException as e:    # base class of all of the above
    print("request failed", e)

```

15.3 Authentication patterns

```

import os, requests
from requests.auth import HTTPBasicAuth

# 1) API key in header or query
h = {"X-API-Key": os.environ["API_KEY"]}
# 2) Bearer token (JWT / OAuth access token)
h = {"Authorization": f"Bearer {os.environ['API_TOKEN']}"}
# 3) HTTP Basic
r = requests.get("https://api.example.com/me", auth=HTTPBasicAuth("user", "pass"), timeout=10)

# 4) OAuth2 client-credentials flow: exchange id/secret for a short-lived token
def get_token():
    r = requests.post("https://auth.example.com/oauth/token",
                      data={"grant_type": "client_credentials",
                            "client_id": os.environ["CLIENT_ID"],
                            "client_secret": os.environ["CLIENT_SECRET"]},
                      timeout=15)
    r.raise_for_status()
    body = r.json()
    return body["access_token"], body.get("expires_in", 3600)
# Cache the token and refresh shortly before expires_in; on 401 refresh once and retry.
# Never hard-code secrets: env vars, .env (python-dotenv), or a secrets manager.

```

15.4 Session, retries and rate limits

```

import requests, time
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def make_session(token=None):
    s = requests.Session()                # reuses TCP connections, shared headers
    s.headers.update({"Accept": "application/json", "User-Agent": "my-etl/1.0"})
    if token:
        s.headers["Authorization"] = f"Bearer {token}"
    retry = Retry(
        total=5, backoff_factor=1,          # sleeps 1s, 2s, 4s ...
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["GET", "HEAD", "OPTIONS"],    # only idempotent methods
        respect_retry_after_header=True,
    )

```

```

adapter = HTTPAdapter(max_retries=retry)
s.mount("https://", adapter); s.mount("http://", adapter)
return s

# Manual handling of 429 with Retry-After
def get_with_rate_limit(session, url, **kw):
    for attempt in range(5):
        r = session.get(url, timeout=30, **kw)
        if r.status_code == 429:
            wait = int(r.headers.get("Retry-After", 2 * attempt))
            time.sleep(wait)
            continue
        r.raise_for_status()
        return r
    raise RuntimeError("rate limited too many times")

```

15.5 Pagination: the four common styles

Never assume an API returns everything in one call. Read the docs to find which style is used, then loop until there are no more pages. Use **generators** so memory stays flat.

Style	Request looks like	Stop when
Page number	?page=1&per_page=100	empty page, or page > total_pages
Offset / limit	?offset=0&limit=100	fewer than limit items returned
Cursor / token	?cursor=abc (response has next_cursor)	next_cursor missing or null
Link header / next URL	response header Link: <url>; rel="next" or JSON next	no next link

```

# 1) PAGE NUMBER pagination
def fetch_pages(session, url, per_page=100, max_pages=10_000):
    page = 1
    while page <= max_pages:
        r = session.get(url, params={"page": page, "per_page": per_page}, timeout=30)
        r.raise_for_status()
        items = r.json().get("data", [])
        if not items:
            break
        yield from items
        page += 1
    # generator: one record at a time

# 2) OFFSET / LIMIT pagination
def fetch_offset(session, url, limit=100):
    offset = 0
    while True:
        r = session.get(url, params={"offset": offset, "limit": limit}, timeout=30)
        r.raise_for_status()
        items = r.json()["results"]
        yield from items
        if len(items) < limit:
            break
        offset += limit
    # last (short) page

# 3) CURSOR / TOKEN pagination (most robust: stable while data changes)
def fetch_cursor(session, url, page_size=200):
    cursor = None
    while True:
        params = {"limit": page_size}
        if cursor:
            params["cursor"] = cursor
        r = session.get(url, params=params, timeout=30)
        r.raise_for_status()
        body = r.json()
        yield from body["items"]
        cursor = body.get("next_cursor")
        if not cursor:
            break

# 4) NEXT-URL / Link header pagination (GitHub style)
def fetch_links(session, url, params=None):
    while url:
        r = session.get(url, params=params, timeout=30)

```

```

    r.raise_for_status()
    yield from r.json()
    url = r.links.get("next", {}).get("url")      # requests parses Link header
    params = None                                # next URL already contains the query

```

```

# Runnable demo with a FAKE api (no network) to practise the pattern
DATA = [{"id": i} for i in range(1, 26)]          # 25 records

def fake_api(offset=0, limit=10):
    chunk = DATA[offset: offset + limit]
    return {"results": chunk, "total": len(DATA)}

def fetch_all(limit=10):
    offset = 0
    while True:
        body = fake_api(offset, limit)
        yield from body["results"]
        offset += limit
        if offset >= body["total"]:
            break

all_rows = list(fetch_all())
print(len(all_rows), all_rows[0], all_rows[-1])    # 25 {'id': 1} {'id': 25}

```

15.6 End-to-end extract -> transform -> load example

```

import requests, json, csv, logging
from pathlib import Path
log = logging.getLogger("etl.api")

def extract(session, base_url):
    total = 0
    for rec in fetch_cursor(session, f"{base_url}/orders"):    # from 15.5
        total += 1
        yield rec
    log.info("extracted %d records", total)

def transform(rec):
    """Flatten one nested API record and normalise types."""
    return {
        "order_id": int(rec["id"]),
        "customer": (rec.get("customer") or {}).get("name"),    # nested + may be null
        "amount": float(rec.get("amount", 0) or 0),
        "status": (rec.get("status") or "unknown").lower(),
        "created_at": rec["created_at"][:19],                  # trim to seconds
    }

def load_csv(rows, path):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    n = 0
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=["order_id", "customer", "amount", "status", "created_at"])
        w.writeheader()
        for r in rows:
            w.writerow(r); n += 1
    return n

def run(base_url, out="out/orders.csv"):
    s = make_session()    # from 15.4
    rows = (transform(r) for r in extract(s, base_url))    # lazy pipeline, constant memory
    n = load_csv(rows, out)
    log.info("wrote %d rows to %s", n, out)

# Save raw payloads too (bronze layer) so you can re-process without calling the API again:
# Path("raw/orders_2026-09-24.jsonl").write_text("\n".join(json.dumps(r) for r in records))

```

15.7 Incremental loads, idempotency, JSON to DataFrame

```
import json, pandas as pd
from pathlib import Path

# Incremental extraction with a watermark (only fetch what changed)
STATE = Path("state.json")
def get_watermark(default="1970-01-01T00:00:00Z"):
    return json.loads(STATE.read_text())["last_updated"] if STATE.exists() else default
def set_watermark(value):
    STATE.write_text(json.dumps({"last_updated": value}))
# params={"updated_since": get_watermark()} ... then set_watermark(max(r["updated_at"]))
# Update the watermark ONLY after the load succeeded, so a failed run is safely re-run.

# Idempotent loads: re-running must not create duplicates -> MERGE/upsert on a business key
# or dedupe by key before insert: latest = {r["id"]: r for r in records}.values()

# Nested JSON -> DataFrame
records = [{"id": 1, "user": {"name": "A", "geo": {"city": "Pune"}}, "items": [{"sku": "x", "q": 2}]]
df = pd.json_normalize(records, sep="_") # user_name, user_geo_city, items
lines = pd.json_normalize(records, record_path="items", meta=["id", ["user", "name"]])
print(df.columns.tolist())
print(lines) # one row per item with parent id
```

15.8 Concurrency for API calls

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch_one(session, url):
    r = session.get(url, timeout=20)
    r.raise_for_status()
    return r.json()

urls = [f"https://api.example.com/items/{i}" for i in range(50)]
results, errors = [], []
with ThreadPoolExecutor(max_workers=8) as pool: # I/O bound -> threads are fine
    futures = {pool.submit(fetch_one, make_session(), u): u for u in urls}
    for fut in as_completed(futures):
        try:
            results.append(fut.result())
        except Exception as e:
            errors.append((futures[fut], str(e))) # collect, do not crash the batch
# Respect the provider's rate limit: keep max_workers small, back off on 429.

# asyncio + httpx/aiohttp variant
import asyncio, httpx
async def fetch_async(urls):
    async with httpx.AsyncClient(timeout=20) as client:
        resps = await asyncio.gather(*(client.get(u) for u in urls))
    return [r.json() for r in resps]
# asyncio.run(fetch_async(urls))
```

15.9 Testing API code without the network

```
from unittest.mock import patch, MagicMock

def get_user(uid):
    import requests
    r = requests.get(f"https://api.example.com/users/{uid}", timeout=10)
    r.raise_for_status()
    return r.json()

def test_get_user():
    fake = MagicMock(); fake.json.return_value = {"id": 1}; fake.raise_for_status.return_value = None
    with patch("requests.get", return_value=fake) as m:
        assert get_user(1) == {"id": 1}
        m.assert_called_once()
# For richer tests use the `responses` or `requests-mock` libraries.
```

16. Python for Data Engineering

16.1 pandas essentials

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"id": [1, 2, 3, 3], "dept": ["IT", "HR", "IT", "IT"],
                  "sal": [100, 80, None, 120], "joined": ["2024-01-05", "2023-03-10", "2022-07-01",
                  "2022-07-01"]})
df.info(); print(df.shape, df.dtypes)           # structure and types
print(df.head(2), df.describe())
print(df["sal"], df[["id", "sal"]], df.loc[0, "sal"], df.iloc[0:2, 0:2]) # label vs position
print(df[df["sal"] > 90], df.query("dept == 'IT' and sal > 90"))          # filtering
df["joined"] = pd.to_datetime(df["joined"])      # cast column type
df["sal"] = df["sal"].fillna(df["sal"].median()) # missing values (dropna() removes)
df["bonus"] = np.where(df["sal"] > 100, 10, 5)   # vectorised if/else
df["year"] = df["joined"].dt.year               # date parts
df["dept"] = df["dept"].str.lower().str.strip() # string ops on a column
df = df.drop_duplicates(subset=["id"], keep="last") # de-dupe by key
print(df.groupby("dept")["sal"].agg(["count", "mean", "max"]))           # group-by
print(df.groupby("dept").agg(total=("sal", "sum"), people=("id", "nunique")))
depts = pd.DataFrame({"dept": ["it", "hr"], "head": ["Zed", "Yan"]})
out = df.merge(depts, on="dept", how="left")    # join: inner/left/right/outer
print(pd.concat([df, df], df.sort_values("sal", ascending=False)))
print(df.isna().sum())                        # null count per column
df.rename(columns={"sal": "salary"}).to_csv("o.csv", index=False)
# Also: to_parquet, read_json, read_sql, pivot_table, apply, astype, value_counts, rolling
```

16.2 Databases with sqlite3 (DB-API pattern used by every driver)

```
import sqlite3
conn = sqlite3.connect("demo.db")              # ":memory:" for tests
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS t (id INTEGER PRIMARY KEY, name TEXT, amt REAL)")

cur.execute("INSERT INTO t (name, amt) VALUES (?, ?)", ("Asha", 9.5))    # bind params
cur.executemany("INSERT INTO t (name, amt) VALUES (?, ?)", [("B", 1.0), ("C", 2.0)]) # bulk
conn.commit()                          # transactions: commit / rollback

# NEVER build SQL with f-strings from user input (SQL injection). Use placeholders.
cur.execute("SELECT * FROM t WHERE amt > ?", (1.5,))
print(cur.fetchone(), cur.fetchall())    # one row / all remaining rows
conn.row_factory = sqlite3.Row           # rows accessible by column name
for row in conn.execute("SELECT name, amt FROM t"):
    print(row["name"], row["amt"])

try:
    with conn:
        conn.execute("INSERT INTO t (id, name) VALUES (1, 'dup')")
except sqlite3.IntegrityError as e:
    print("constraint failed:", e)
conn.close()
# Other drivers use the same shape: psycopg2 (Postgres), pyodbc (SQL Server), pymysql,
# snowflake.connector: connect -> cursor -> execute -> fetch -> close.
```

16.3 Snowflake connector and bulk loading (data migration pattern)

```
import os
import snowflake.connector
import pandas as pd
from snowflake.connector.pandas_tools import write_pandas

conn = snowflake.connector.connect(
    account=os.environ["SF_ACCOUNT"], user=os.environ["SF_USER"],
    password=os.environ["SF_PASSWORD"], # or key-pair auth / SSO in production
```

```

warehouse="LOAD_WH", database="RAW", schema="PUBLIC", role="LOADER",
)
try:
    cur = conn.cursor()
    cur.execute("SELECT CURRENT_VERSION()")
    print(cur.fetchone())
    cur.execute("SELECT id, name FROM customers WHERE created_at >= %s", ("2026-01-01",)) # binds
    rows = cur.fetchall()

    df = pd.DataFrame({"ID": [1, 2], "NAME": ["A", "B"]}) # column names UPPERCASE
    ok, nchunks, nrows, _ = write_pandas(conn, df, "CUSTOMERS", auto_create_table=True)
    print(ok, nchunks)

    # File based bulk load (fastest for large data): stage then COPY
    cur.execute("PUT file:///data/orders_*.csv @%ORDERS AUTO_COMPRESS=TRUE")
    cur.execute("COPY INTO ORDERS FROM @%ORDERS FILE_FORMAT=(TYPE=CSV SKIP_HEADER=1) ON_ERROR=ABORT_STATEMENT")
finally:
    conn.close() # always release
# Prefer many rows per statement / COPY over row-by-row INSERTs in a loop.

```

16.4 Migration and validation patterns

```

import hashlib, logging
log = logging.getLogger("migrate")

# 1) Row-count and aggregate reconciliation between source and target
def reconcile(src_count, tgt_count, src_sum, tgt_sum, tol=1e-6):
    problems = []
    if src_count != tgt_count:
        problems.append(f"count mismatch {src_count} vs {tgt_count}")
    if abs(src_sum - tgt_sum) > tol:
        problems.append(f"sum mismatch {src_sum} vs {tgt_sum}")
    return problems

# 2) Row-level checksum to detect changed rows
def row_hash(row: dict, cols):
    joined = "|".join(" " if row[c] is None else str(row[c]) for c in cols)
    return hashlib.md5(joined.encode("utf-8")).hexdigest()

# 3) Keys present in source but not in target (set difference)
def missing_keys(source_keys, target_keys):
    return set(source_keys) - set(target_keys)

# 4) Data quality checks
def dq_checks(rows):
    issues = {"null_id": 0, "neg_amt": 0, "dupe_id": 0}
    seen = set()
    for r in rows:
        if r.get("id") is None: issues["null_id"] += 1
        elif r["id"] in seen: issues["dupe_id"] += 1
        else: seen.add(r["id"])
        if (r.get("amt") or 0) < 0: issues["neg_amt"] += 1
    return issues

# 5) Type mapping dictionary for schema migration (source type -> target type)
TYPE_MAP = {"NUMBER": "NUMBER", "VARCHAR2": "VARCHAR", "DATE": "TIMESTAMP_NTZ", "CLOB": "VARCHAR"}
def map_column(name, src_type):
    return f"{name} {TYPE_MAP.get(src_type.upper(), 'VARCHAR')}"
print(map_column("created", "date")) # created TIMESTAMP_NTZ

# 6) Chunked, restartable loads with logging of progress
def load_in_batches(rows, insert_fn, size=10_000):
    total = 0
    for i in range(0, len(rows), size):
        insert_fn(rows[i:i + size])
        total += len(rows[i:i + size])
    log.info("loaded %d/%d", total, len(rows))

```

16.5 Concurrency: threads vs processes vs asyncio

Model	Use for	Notes
<code>threading</code> / <code>ThreadPoolExecutor</code>	I/O bound (API calls, files, DB)	GIL prevents parallel CPU work in threads
<code>multiprocessing</code> / <code>ProcessPoolExecutor</code>	CPU bound (parsing, hashing, transforms)	separate processes, pickling overhead
<code>asyncio</code>	many concurrent network calls	single thread, cooperative, needs async libraries

The **GIL** (Global Interpreter Lock) lets only one thread execute Python bytecode at a time in CPython; it is released during I/O, so threads help I/O-bound work only.

```
from concurrent.futures import ProcessPoolExecutor
def heavy(n): return sum(i * i for i in range(n))
if __name__ == "__main__":
    # required for multiprocessing on Windows/macOS
    with ProcessPoolExecutor() as pool:
        print(list(pool.map(heavy, [10**6, 10**6, 10**6])))

import asyncio
async def work(i):
    await asyncio.sleep(0.1)          # non-blocking wait
    return i
async def main():
    return await asyncio.gather(*(work(i) for i in range(5)))
print(asyncio.run(main()))           # [0, 1, 2, 3, 4] in ~0.1s total
```

16.6 Project hygiene: entry point, config, tests, environments

```
# etl.py: standard layout
import argparse, logging, os, sys

def main(argv=None) -> int:
    args = argparse.ArgumentParser().parse_args(argv)
    logging.basicConfig(level=os.getenv("LOG_LEVEL", "INFO"))
    try:
        # run pipeline ...
        return 0
    except Exception:
        logging.getLogger(__name__).exception("pipeline failed")
        return 1

if __name__ == "__main__":
    # runs only when executed, not when imported
    sys.exit(main())
```

```
# environments and tooling (shell)
python -m venv .venv && source .venv/bin/activate  # isolated environment
pip install requests pandas pytest                # install
pip freeze > requirements.txt                      # pin versions
pip install -r requirements.txt
pytest -q                                           # run tests
python -m pytest tests/test_x.py::test_name -k keyword
python -m pdb script.py                           # debugger ; breakpoint() inside code
```

```
# pytest example
import pytest
def add(a, b): return a + b

def test_add():
    assert add(2, 3) == 5

@pytest.mark.parametrize("a,b,exp", [(1, 1, 2), (0, 0, 0), (-1, 1, 0)])
def test_add_many(a, b, exp):
    assert add(a, b) == exp

def test_error():
    with pytest.raises(ZeroDivisionError):
        1 / 0

@pytest.fixture
def sample_rows():
    return [{"id": 1}, {"id": 2}]
def test_rows(sample_rows):
    assert len(sample_rows) == 2
```

17. DuckDB: Connections, SQL, and Python Functions

17.1 What DuckDB is and when to use it

DuckDB is an **embedded, in-process, columnar analytical (OLAP) database**: no server, one `pip install`, runs inside your Python process, vectorised execution, and it queries CSV / Parquet / JSON files and pandas DataFrames directly with full SQL. Think "SQLite for analytics".

Need	Use DuckDB when	Prefer something else when
Analytics on files	Parquet / CSV / JSON of GBs on one machine	Data is many TB: use Snowflake / Spark
Data engineering	Validation, reconciliation, transforms, prototyping warehouse SQL locally	Many concurrent writers / OLTP: use Postgres
Testing	Fast in-memory tests of SQL logic	Need multi-user server access
vs pandas	Larger than memory, SQL joins/aggregations, window functions	Small data and simple reshaping
vs SQLite	Aggregations, columnar scans, Parquet	Tiny transactional row lookups

Install: `pip install duckdb`. Check version: `duckdb.__version__`.

17.2 Connecting

```
import duckdb

# 1) In-memory database (data vanishes when the connection closes)
con = duckdb.connect() # same as duckdb.connect(":memory:")

# 2) Persistent database file (created if missing; single file)
con = duckdb.connect("warehouse.duckdb")

# 3) Read-only (allows several processes to read the same file at once)
ro = duckdb.connect("warehouse.duckdb", read_only=True)

# 4) With configuration
con = duckdb.connect("warehouse.duckdb", config={"threads": 4, "memory_limit": "2GB"})

# 5) Context manager closes the connection for you
with duckdb.connect("warehouse.duckdb") as c:
    print(c.execute("SELECT 42").fetchone()) # (42,)

# 6) Module-level default connection (global in-memory DB, handy in notebooks)
duckdb.sql("CREATE TABLE t AS SELECT 1 AS id")
print(duckdb.sql("SELECT * FROM t").fetchall()) # [(1,)]

con.close()
```

Concurrency rules: one process may **write** (read-write connection) at a time; many processes may open the file `read_only=True`. Inside one process, use `con.cursor()` to give each thread its own connection to the same database. Connections are not thread-safe when shared.

17.3 Running queries and getting results

```
import duckdb
con = duckdb.connect()

con.execute("CREATE TABLE emp (id INTEGER, name VARCHAR, dept VARCHAR, sal DOUBLE)")
con.execute("INSERT INTO emp VALUES (1, 'Asha', 'IT', 100), (2, 'Bob', 'HR', 80), (3, 'Cy', 'IT', 120)")

# execute() runs the SQL and returns the connection; then fetch:
cur = con.execute("SELECT * FROM emp ORDER BY id")
print(cur.fetchone())      # (1, 'Asha', 'IT', 100.0) one row as tuple
print(cur.fetchmany(1))    # next 1 row -> [(2, 'Bob', 'HR', 80.0)]
print(cur.fetchall())      # remaining rows as list of tuples
print(cur.description)     # column metadata: [('id', 'NUMBER', ...), ...]
print([d[0] for d in cur.description]) # column names

# Result formats
con.sql("SELECT * FROM emp").show()      # pretty printed table in console
df = con.sql("SELECT * FROM emp").df()   # pandas DataFrame (also fetchdf())
arrow = con.sql("SELECT * FROM emp").arrow() # PyArrow table (fetch_arrow_table())
pl = con.sql("SELECT * FROM emp").pl()    # Polars DataFrame
np_ = con.sql("SELECT * FROM emp").fetchnumpy() # dict of NumPy arrays

# Parameterised queries: NEVER build SQL with f-strings from user input
con.execute("SELECT * FROM emp WHERE dept = ? AND sal > ?", ["IT", 90]).fetchall() # positional
con.execute("SELECT * FROM emp WHERE dept = $d", {"d": "HR"}).fetchall()          # named
con.executemany("INSERT INTO emp VALUES (?, ?, ?, ?)", [[4, "Di", "HR", 70], [5, "Ed", "IT", 90]])

# sql() vs execute(): sql() returns a lazy Relation (composable), execute() runs immediately
rel = con.sql("SELECT dept, sal FROM emp") # nothing has run yet
print(rel.columns, rel.types, rel.shape)  # ['dept', 'sal'] [VARCHAR, DOUBLE] ...
```

17.4 Reading files directly (CSV, Parquet, JSON)

```
import duckdb
con = duckdb.connect()

# Query a file by name, no load step needed
con.sql("SELECT * FROM 'orders.csv' LIMIT 5").show()
con.sql("SELECT count(*) FROM 'data/2026/*.parquet'").show() # glob patterns

# Explicit readers with options
con.sql("""
    SELECT * FROM read_csv('orders.csv',
        header = true, delimiter = ',', quote = '"',
        columns = {'id': 'INTEGER', 'amt': 'DOUBLE', 'ts': 'TIMESTAMP'},
        dateformat = '%Y-%m-%d', ignore_errors = true, skip = 0)
""")
con.sql("SELECT * FROM read_csv('big.csv', all_varchar = true, sample_size = -1)") # safest typing

# Parquet (columnar: reads only needed columns and row groups)
con.sql("SELECT id, amt FROM read_parquet('data/*.parquet', filename = true, union_by_name = true)")
con.sql("SELECT * FROM read_parquet('lake/year=*/month=*/*.parquet', hive_partitioning = true)")
con.sql("SELECT * FROM parquet_schema('a.parquet')") # inspect schema
con.sql("SELECT * FROM parquet_metadata('a.parquet')") # row groups, stats

# JSON and JSON Lines
con.sql("SELECT * FROM read_json_auto('events.json')")
con.sql("SELECT * FROM read_json('events.jsonl', format = 'newline_delimited')")
con.sql("SELECT payload->'$.user.name' AS name FROM read_json_auto('e.json')") # JSON path

# Python-side readers return relations
rel_csv = con.read_csv("orders.csv")
rel_pq = con.read_parquet("a.parquet")
rel_js = con.read_json("e.json")
```

17.5 Writing files and exporting

```
import duckdb
con = duckdb.connect()
con.sql("CREATE TABLE sales AS SELECT range AS id, range % 5 AS region, random() AS amt FROM range(1000)")

# COPY ... TO writes a table or ANY query result
con.sql("COPY sales TO 'sales.parquet' (FORMAT PARQUET, COMPRESSION zstd)")
con.sql("COPY (SELECT * FROM sales WHERE amt > 0.5) TO 'hi.csv' (HEADER, DELIMITER ',')")
con.sql("COPY sales TO 'lake' (FORMAT PARQUET, PARTITION_BY (region), OVERWRITE_OR_IGNORE)") # hive folders
con.sql("COPY sales TO 'sales.json' (FORMAT JSON, ARRAY true)")

# COPY ... FROM loads a file into an existing table
con.sql("CREATE TABLE t2 (id BIGINT, region BIGINT, amt DOUBLE)")
con.sql("COPY t2 FROM 'sales.parquet' (FORMAT PARQUET)")

# Relation API writers
con.sql("SELECT * FROM sales").write_parquet("s.parquet")
con.sql("SELECT * FROM sales").write_csv("s.csv")

# Whole database export/import (portable backup)
con.sql("EXPORT DATABASE 'backup_dir' (FORMAT PARQUET)")
con.sql("IMPORT DATABASE 'backup_dir'")
```

17.6 pandas, Arrow and Polars integration

```
import duckdb, pandas as pd
df = pd.DataFrame({"id": [1, 2, 3], "grp": ["a", "b", "a"], "v": [10.0, 20.0, 30.0]})

# Replacement scan: DuckDB finds the DataFrame variable by NAME in your Python scope
print(duckdb.sql("SELECT grp, sum(v) AS total FROM df GROUP BY grp").df())

con = duckdb.connect()
con.register("orders_df", df) # explicit registration under a chosen name
print(con.sql("SELECT count(*) FROM orders_df").fetchone())
con.unregister("orders_df")

con.execute("CREATE TABLE persisted AS SELECT * FROM df") # materialise into a real table
rel = con.from_df(df) # DataFrame -> relation
# PyArrow tables and Polars DataFrames work the same way (variable name in SQL or register).

# Typical hybrid pattern: heavy SQL in DuckDB, final touches in pandas
out = duckdb.sql("""
    SELECT grp, avg(v) AS avg_v FROM df GROUP BY grp ORDER BY grp
""").df()
```

17.7 Relational API (method chaining instead of SQL)

```
import duckdb
con = duckdb.connect()
rel = con.sql("SELECT * FROM (VALUES (1, 'IT', 100), (2, 'HR', 80), (3, 'IT', 120)) t(id, dept, sal)")

print(rel.filter("sal > 90").project("id, dept").order("id").limit(5).fetchall())
print(rel.aggregate("dept, sum(sal) AS total, count(*) AS n", "dept").order("dept").df())
print(rel.distinct().count("id").fetchall())
other = con.sql("SELECT * FROM (VALUES ('IT', 'Zed'), ('HR', 'Yan')) h(dept, head)")
print(rel.join(other, "dept").project("id, head").fetchall()) # join on shared column name
print(rel.describe()) # quick statistics per column
rel.to_table("emp_copy") # save relation as a table
rel.create_view("emp_view", replace=True) # or as a view
print(rel.alias, rel.columns) # relation metadata
```

Relations are lazy and composable; nothing runs until you call a terminal method such as `fetchall()`, `df()`, `show()`, `write_parquet()`.

17.8 SQL features that make DuckDB pleasant (and are asked about)

```
-- Inspect
SHOW TABLES;                                DESCRIBE emp;                                SUMMARIZE emp;
PRAGMA table_info('emp');                     EXPLAIN SELECT * FROM emp; EXPLAIN ANALYZE SELECT * FROM emp;

-- Friendly SQL
SELECT * EXCLUDE (sal) FROM emp;               -- all columns except some
SELECT * REPLACE (upper(name) AS name) FROM emp; -- transform one column, keep the rest
SELECT COLUMNS('.*al') FROM emp;              -- columns matching a regex
SELECT dept, count(*), avg(sal) FROM emp GROUP BY ALL; -- group by every non-aggregate column
SELECT * FROM emp ORDER BY ALL;
FROM emp SELECT name;                          -- FROM-first syntax
SELECT sal::INTEGER, TRY_CAST('x' AS INTEGER) FROM emp; -- :: cast, TRY_CAST returns NULL on failure

-- QUALIFY filters window results (dedupe, top-N per group)
SELECT * FROM emp
QUALIFY row_number() OVER (PARTITION BY dept ORDER BY sal DESC) = 1;

-- Window functions, CTEs, filters on aggregates
WITH ranked AS (
  SELECT *, rank() OVER (PARTITION BY dept ORDER BY sal DESC) AS rk,
         sum(sal) OVER (PARTITION BY dept) AS dept_total,
         lag(sal) OVER (ORDER BY id) AS prev_sal
  FROM emp)
SELECT dept, count(*) FILTER (WHERE sal > 90) AS high_paid FROM ranked GROUP BY dept;

-- Sampling and series
SELECT * FROM emp USING SAMPLE 10%;           SELECT * FROM emp USING SAMPLE 100 ROWS;
SELECT * FROM range(5);                       SELECT * FROM generate_series(DATE '2026-01-01', DATE '2026-01-05',
  INTERVAL 1 DAY);

-- Lists, structs, maps, UNNEST
SELECT [1,2,3] AS l, {'a': 1, 'b': 'x'} AS s, MAP {'k': 1} AS m;
SELECT unnest([1,2,3]);                       SELECT list_transform([1,2,3], x -> x * 2);
SELECT [x * 2 FOR x IN [1,2,3] IF x > 1];      -- list comprehension (newer versions)
SELECT dept, list(name) AS names, string_agg(name, ',') FROM emp GROUP BY dept;
SELECT s.a FROM (SELECT {'a': 1} AS s);        -- struct field access

-- PIVOT / UNPIVOT
PIVOT emp ON dept USING sum(sal);
UNPIVOT wide ON jan, feb, mar INTO NAME month VALUE amount;

-- ASOF join (match nearest earlier timestamp: prices, sessions)
SELECT * FROM trades t ASOF JOIN quotes q ON t.sym = q.sym AND t.ts >= q.ts;

-- Handy aggregates
SELECT median(sal), quantile_cont(sal, 0.9), mode(dept), arg_max(name, sal),
       approx_count_distinct(id), count(DISTINCT dept) FROM emp;
```

17.9 Tables, constraints, upserts, transactions

```
import duckdb
con = duckdb.connect("demo.duckdb")

con.execute("""
  CREATE OR REPLACE TABLE customers (
    id INTEGER PRIMARY KEY, name VARCHAR NOT NULL,
    email VARCHAR UNIQUE, created TIMESTAMP DEFAULT current_timestamp)
""")
con.execute("CREATE SEQUENCE IF NOT EXISTS seq_id START 1")
con.execute("CREATE VIEW IF NOT EXISTS v_names AS SELECT name FROM customers")
con.execute("CREATE INDEX IF NOT EXISTS ix_name ON customers(name)") # rarely needed: columnar scans are fast

# Upsert (idempotent load): insert new, update existing by key
con.execute("""
  INSERT INTO customers (id, name, email) VALUES (1, 'Asha', 'a@x.com')
  ON CONFLICT (id) DO UPDATE SET name = excluded.name, email = excluded.email
""")
con.execute("INSERT OR REPLACE INTO customers (id, name, email) VALUES (1, 'Asha B', 'a@x.com')")
con.execute("INSERT OR IGNORE INTO customers (id, name, email) VALUES (1, 'dup', 'd@x.com')")
# MERGE INTO exists in recent versions (1.4+): MERGE INTO t USING s ON ... WHEN MATCHED ...
```

```

con.execute("UPDATE customers SET name = 'A' WHERE id = 1")
con.execute("DELETE FROM customers WHERE id = 99")
con.execute("ALTER TABLE customers ADD COLUMN city VARCHAR")
con.execute("ALTER TABLE customers RENAME COLUMN city TO town")
con.execute("DROP TABLE IF EXISTS scratch")

# Transactions: all or nothing
con.begin()
try:
    con.execute("INSERT INTO customers (id, name) VALUES (2, 'Bob')")
    con.execute("INSERT INTO customers (id, name) VALUES (2, 'Dup')") # violates PRIMARY KEY
    con.commit()
except duckdb.ConstraintException as e:
    con.rollback()
    print("rolled back:", e)
con.execute("CHECKPOINT") # flush WAL into the database file

```

17.10 Extensions, remote files, attaching other databases

```

import duckdb
con = duckdb.connect()

con.sql("INSTALL httpfs") # download once
con.sql("LOAD httpfs") # activate for this connection
# Read straight from cloud storage or HTTPS
con.sql("SELECT count(*) FROM read_parquet('https://example.com/data.parquet')")
con.sql("""
    CREATE SECRET my_s3 (TYPE S3, KEY_ID 'AKIA...', SECRET 'xxx', REGION 'us-east-1')
    """) # prefer env / IAM roles over literals
con.sql("SELECT * FROM read_parquet('s3://my-bucket/lake/*.parquet') LIMIT 10")
con.sql("COPY (SELECT 1 AS x) TO 's3://my-bucket/out/x.parquet' (FORMAT PARQUET)")

# Attach other databases and query across them
con.sql("ATTACH 'other.duckdb' AS other")
con.sql("ATTACH 'legacy.sqlite' AS lite (TYPE sqlite)") # sqlite extension
con.sql("ATTACH 'dbname=app host=db user=u password=p' AS pg (TYPE postgres, READ_ONLY)")
con.sql("SELECT * FROM pg.public.orders LIMIT 5")
con.sql("CREATE TABLE local_copy AS SELECT * FROM pg.public.orders") # pull into DuckDB
con.sql("DETACH other")

print(con.sql("SELECT extension_name, loaded FROM duckdb_extensions() WHERE loaded"))

```

Other useful extensions: `json`, `parquet`, `icu` (time zones), `spatial`, `excel`, `fts` (full-text search), `delta`, `iceberg`, `mysql`, `postgres`, `sqlite`, `httpfs`. Some are bundled and auto-load; others need `INSTALL` then `LOAD`.

17.11 Settings, performance, and out-of-core processing

```

import duckdb
con = duckdb.connect()
con.sql("SET threads = 4")
con.sql("SET memory_limit = '4GB'")
con.sql("SET temp_directory = '/tmp/duck_spill'") # spill to disk when data exceeds memory
con.sql("SET preserve_insertion_order = false") # lowers memory for big COPY / exports
con.sql("SET enable_progress_bar = true")
print(con.sql("SELECT current_setting('threads')").fetchone())
print(con.sql("SELECT * FROM duckdb_settings() WHERE name = 'memory_limit'").fetchall())
con.sql("PRAGMA database_size") # storage information
con.sql("PRAGMA version")

```

Performance tips: store analytics data as **Parquet** (compressed, columnar, predicate and projection push-down); select only needed columns; filter early; use `read_parquet` globs plus hive partition pruning; convert repeated CSV reads into a table or Parquet once; use `EXPLAIN ANALYZE` to see where time goes; avoid pulling millions of rows into Python (`fetchall`), keep work in SQL and export with `COPY`.

17.12 Python UDFs, cursors, threads, and errors

```
import duckdb
from duckdb.typing import VARCHAR, INTEGER    # newer releases also expose duckdb.sqltypes
con = duckdb.connect()

# Register a Python function as a SQL function
def clean(s: str) -> str:
    return s.strip().title() if s else s
con.create_function("clean", clean, [VARCHAR], VARCHAR)
print(con.sql("SELECT clean(' asha ')").fetchone())    # ('Asha',)
# Native SQL is far faster than Python UDFs: use UDFs only when SQL cannot do it.
con.remove_function("clean")

# Macros (reusable SQL, no Python overhead)
con.sql("CREATE MACRO add_tax(x, rate := 0.18) AS x * (1 + rate)")
print(con.sql("SELECT add_tax(100), add_tax(100, rate := 0.05)").fetchall())    # [(118.0, 105.0)]
con.sql("CREATE MACRO top_rows(n) AS TABLE SELECT * FROM range(n)")

# One cursor per thread against the same database
from concurrent.futures import ThreadPoolExecutor
def job(i):
    cur = con.cursor()    # independent connection to the same DB
    return cur.execute("SELECT ? * 2", [i]).fetchone()[0]
with ThreadPoolExecutor(4) as pool:
    print(list(pool.map(job, range(4))))    # [0, 2, 4, 6]

# Exception hierarchy: duckdb.Error is the base
try:
    con.sql("SELECT * FROM missing_table")
except duckdb.CatalogException as e:    # table / column / function not found
    print("catalog:", e)
try:
    con.sql("SELEC 1")
except duckdb.ParserException as e:    # SQL syntax error
    print("syntax:", e)
try:
    con.sql("SELECT CAST('abc' AS INTEGER)").fetchall()
except duckdb.ConversionException as e:    # bad cast
    print("conversion:", e)
try:
    con.sql("SELECT nope FROM (SELECT 1 AS a)")
except duckdb.BinderException as e:    # unknown column, ambiguous name
    print("binder:", e)
# Others: IOException (file/network), ConstraintException, InvalidInputException,
# OutOfMemoryException, TransactionException; catch duckdb.Error for everything.
```

17.13 Date, string and conversion functions you will use daily

```
-- Dates and time
SELECT current_date, current_timestamp, now();
SELECT date_trunc('month', TIMESTAMP '2026-09-24 10:30:00');    -- 2026-09-01 00:00:00
SELECT date_diff('day', DATE '2026-01-01', DATE '2026-09-24');    -- 266
SELECT date_add(DATE '2026-01-31', INTERVAL 1 MONTH), DATE '2026-09-24' + INTERVAL 7 DAY;
SELECT year(d), month(d), dayofweek(d), strftime(d, '%Y-%m-%d'), strftime('24/09/2026', '%d/%m/%Y') FROM (SELECT
    DATE '2026-09-24' AS d);
SELECT epoch(TIMESTAMP '2026-09-24'), to_timestamp(1758700000);

-- Strings
SELECT upper('a'), lower('A'), trim(' x '), length('abc'), substr('abcdef', 2, 3), replace('a-b', '-', '_');
SELECT split_part('a,b,c', ',', 2), string_split('a,b,c', ','), concat('a', 'b'), 'a' || 'b';
SELECT regexp_matches('abc123', '\d+'), regexp_extract('id=42', 'id=(\d+)', 1), regexp_replace('a b', '\s+', ' ', 'g');
SELECT contains('hello', 'ell'), starts_with('hello', 'he'), lpad('7', 3, '0'), md5('x'), sha256('x');

-- Null handling and conditionals
SELECT coalesce(NULL, 'default'), nullif(0, 0), ifnull(NULL, 1), CASE WHEN 1 > 0 THEN 'y' ELSE 'n' END;
SELECT typeof(1), typeof(1.5), typeof('a'), typeof(NULL);
```

17.14 End-to-end example: reconcile a source extract with a target (data migration)

```
import duckdb

con = duckdb.connect()                                # in-memory scratch DB
con.sql("CREATE VIEW src AS SELECT * FROM read_parquet('source/*.parquet')")
con.sql("CREATE VIEW tgt AS SELECT * FROM read_csv('target_export.csv', header = true)")

# 1) Row count and aggregate checks
print(con.sql("""
    SELECT (SELECT count(*) FROM src) AS src_rows,
           (SELECT count(*) FROM tgt) AS tgt_rows,
           (SELECT sum(amount) FROM src) AS src_sum,
           (SELECT sum(amount) FROM tgt) AS tgt_sum
""").df())

# 2) Keys missing on either side (anti joins)
missing_in_tgt = con.sql("SELECT s.id FROM src s ANTI JOIN tgt t ON s.id = t.id")
extra_in_tgt = con.sql("SELECT t.id FROM tgt t ANTI JOIN src s ON s.id = t.id")

# 3) Column-level differences for matching keys
diffs = con.sql("""
    SELECT s.id, s.amount AS src_amt, t.amount AS tgt_amt
    FROM src s JOIN tgt t USING (id)
    WHERE s.amount IS DISTINCT FROM t.amount
""")

# 4) Duplicates in target, keep the latest row per key
dedup = con.sql("""
    SELECT * FROM tgt QUALIFY row_number() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1
""")

# 5) Data-quality profile
print(con.sql("SELECT count(*) - count(id) AS null_ids, count(DISTINCT id) AS distinct_ids FROM
    tgt").fetchone())
con.sql("SUMMARIZE tgt").show()

# 6) Persist evidence for the migration sign-off
con.sql("COPY (SELECT * FROM diffs) TO 'reports/diffs.parquet' (FORMAT PARQUET)")
con.sql("COPY (SELECT * FROM missing_in_tgt) TO 'reports/missing.csv' (HEADER)")
con.close()
```

17.15 DuckDB interview cheat sheet

Question	Short answer
What is DuckDB?	In-process columnar OLAP database with vectorised execution; embedded like SQLite, built for analytics.
How is it different from SQLite?	Columnar and vectorised (fast scans / aggregations) vs row-oriented and OLTP-focused.
How is it different from Snowflake?	Single-node, embedded, no separate compute/storage layer or concurrency service; great for local, dev, testing and mid-size data.
In-memory vs persistent?	<code>connect()</code> / <code>":memory:"</code> vs <code>connect("file.duckdb")</code> .
Can multiple processes use one file?	One read-write process, or many <code>read_only=True</code> processes.
How do I query a pandas DataFrame?	Reference the variable by name in SQL, or <code>con.register("name", df)</code> .
How do I get results as pandas / Arrow?	<code>.df()</code> / <code>.arrow()</code> on the result or relation.
Why Parquet?	Columnar, compressed, push-down filters and projections, schema included.
<code>execute()</code> vs <code>sql()</code> ?	<code>execute</code> runs now and returns the connection; <code>sql</code> returns a lazy relation.
How do I avoid SQL injection?	Bind parameters with <code>?</code> or <code>\$name</code> , never string formatting.
What if data is bigger than memory?	Streaming operators plus disk spilling via <code>temp_directory</code> and <code>memory_limit</code> .
How do I read from S3?	<code>INSTALL https;</code> <code>LOAD https;</code> create a secret, then <code>read_parquet('s3://...')</code> .
What are QUALIFY / EXCLUDE / GROUP BY ALL?	Filter window results / drop columns from <code>*</code> / auto group by non-aggregates.
How do I do an upsert?	<code>INSERT ... ON CONFLICT DO UPDATE</code> or <code>INSERT OR REPLACE</code> (and <code>MERGE INTO</code> in newer versions).

18. Commonly Used Python Functions (with explanations)

18.1 Built-in functions: numbers and aggregation

Function	What it does	Example -> result
<code>len(x)</code>	number of items	<code>len("abc")</code> -> 3
<code>sum(it, start=0)</code>	adds items	<code>sum([1,2,3])</code> -> 6
<code>min/max(it, key=, default=)</code>	smallest / largest	<code>max(["a", "bbb"], key=len)</code> -> 'bbb'
<code>abs(x)</code>	absolute value	<code>abs(-4)</code> -> 4
<code>round(x, n)</code>	rounds (banker's rounding)	<code>round(3.14159, 2)</code> -> 3.14
<code>pow(a, b, mod)</code>	power, optional modulo	<code>pow(2, 10, 1000)</code> -> 24
<code>divmod(a, b)</code>	(quotient, remainder)	<code>divmod(7, 2)</code> -> (3, 1)
<code>range(a, b, s)</code>	lazy integer sequence	<code>list(range(0, 10, 3))</code> -> [0,3,6,9]

```
nums = [4, 9, 1]
print(len(nums), sum(nums), min(nums), max(nums))      # 3 14 1 9
print(max([], default=None))                          # None (avoids ValueError)
print(min(["bb", "a", "ccc"], key=len))               # a
print(sum([[1], [2]], []))                           # [1, 2] (start=[] concatenates)
print(sum(x * x for x in nums) / len(nums))           # mean of squares
```

18.2 Built-ins: iteration and functional

```
names = ["b", "a", "c"]
print(sorted(names), sorted(names, reverse=True))     # new sorted list
print(list(reversed(names)))                          # reversed iterator
print(list(enumerate(names, 1)))                     # [(1,'b'), (2,'a'), (3,'c')]
print(list(zip(names, [1, 2, 3])))                   # pairs items
print(dict(zip(names, range(3))))                   # {'b': 0, 'a': 1, 'c': 2}
a, b = zip(*[(1, "x"), (2, "y")])                   # unzip: a=(1,2) b=('x','y')
print(list(map(str.upper, names)))                   # ['B', 'A', 'C']
print(list(filter(None, [0, 1, "", "a", None])))     # [1, 'a'] None keeps truthy
print(any(n > 5 for n in [1, 9]), all(n > 0 for n in [1, 2])) # True True
print(any([], all([])))                             # False True (empty!)
it = iter(names); print(next(it), next(it))          # b a
print(sorted({"b": 2, "a": 1}.items(), key=lambda kv: kv[1], reverse=True))
```

`any/all` short-circuit and work with generators, so they are cheap on large data. `map/filter/zip/enumerate/reversed` return lazy iterators in Python 3 (wrap in `list()` to see the values).

18.3 Built-ins: types, objects and introspection

```

x = 5
print(type(x), isinstance(x, (int, float)), subclass(bool, int)) # <class 'int'> True True
print(id(x), hash("a") == hash("a"), callable(len)) # identity / hash / callable?
print(dir("s")[:3]) # attribute names of an object
print(hasattr("s", "upper"), getattr("s", "upper")()) # S
class P: pass
p = P(); setattr(p, "k", 1); print(vars(p), p.__dict__) # {'k': 1} {'k': 1}
delattr(p, "k")
print(repr("hi\n"), ascii("e"), format(3.14159, ".2f"), format(255, "b"))
print(chr(65), ord("A"), bin(5), hex(255), oct(8)) # A 65 0b101 0xff 0o10
print(int("ff", 16), float("inf") > 10**100)
print(bool([]), bytes([65, 66]), bytearray(b"ab"), frozenset([1]))
print(help is not None, globals().keys() is not None, locals() is not None)
print(eval("2 + 3 * 4")) # 14 ; NEVER eval untrusted text. Use ast.literal_eval for literals:
import ast
print(ast.literal_eval("{'a': [1, 2]}")) # safe parse of a literal
print(input.__name__, __name__) # __main__ when run directly

```

18.4 String / list / dict method quick recap

```

s = "a,b;c"
print(s.split(","), s.replace(";", ", "), s.upper(), "x".join(["1", "2"]))
lst = [3, 1, 2]
lst.sort(); lst.append(4); lst.extend([5]); lst.insert(0, 0); print(lst.pop(), lst.index(2))
d = {"a": 1}
print(d.get("z", 0), list(d), d.setdefault("b", 2), d.pop("a"), d.items())
st = {1, 2}; st.add(3); print(st | {9}, st & {1}, st - {1})

```

18.5 Standard library functions you will use constantly

```

# math
import math
print(math.floor(2.7), math.ceil(2.1), math.sqrt(16), math.gcd(12, 18), math.factorial(5))
print(math.pi, math.log(math.e), math.log10(1000), math.inf, math.isnan(float("nan")))
print(math.comb(5, 2), math.perm(5, 2), math.hypot(3, 4), math.prod([2, 3, 4])) # 10 20 5.0 24

# random
import random
random.seed(1) # reproducible sequence
print(random.randint(1, 6), random.random(), random.choice(["a", "b"]))
print(random.sample(range(100), 3), random.uniform(1, 2))
l = [1, 2, 3]; random.shuffle(l) # in place

# statistics
import statistics as st
print(st.mean([1, 2, 3, 4]), st.median([1, 3, 2]), st.mode([1, 1, 2]), st.stdev([1, 2, 3, 4]))

# uuid, hashlib, base64, secrets
import uuid, hashlib, base64, secrets
print(uuid.uuid4(), hashlib.sha256(b"abc").hexdigest()[:8], hashlib.md5(b"abc").hexdigest()[:8])
print(base64.b64encode(b"hi"), base64.b64decode("aGk="), secrets.token_hex(8))

# functools
from functools import reduce, lru_cache, partial, wraps, cmp_to_key, cached_property, total_ordering
# copy, textwrap, string, pprint
import copy, textwrap, string, pprint
print(string.ascii_lowercase[:5], string.digits, string.punctuation[:5])
pprint.pprint({"a": [1, 2, {"b": 3}], width=20) # readable nested output
print(textwrap.shorten("hello world this is long", width=15, placeholder="..."))

# decimal, fractions (exact arithmetic)
from decimal import Decimal, ROUND_HALF_UP
print(Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)) # 2.68
from fractions import Fraction
print(Fraction(1, 3) + Fraction(1, 6)) # 1/2

# operator module (fast key functions)
from operator import itemgetter, attrgetter
rows = [{"n": "b", "v": 2}, {"n": "a", "v": 1}]
print(sorted(rows, key=itemgetter("v"))) # sort by field

```

```

# time / date recap
from datetime import datetime, timedelta
print((datetime(2026, 1, 31) + timedelta(days=30)).date()) # 2026-03-02
import calendar; print(calendar.monthrange(2028, 2)) # (1, 29) leap year

# enum + typing + abc + contextlib: see OOP section
# os.walk / shutil / glob / pathlib: see file section
# zipfile
import zipfile
with zipfile.ZipFile("a.zip", "w") as z:
    z.writestr("x.txt", "hello")

```

18.6 One-liners worth memorising

```

lst = [3, 1, 2, 3, 1]
print(list(dict.fromkeys(lst))) # dedupe keep order
print(sorted(set(lst))) # unique sorted
print([lst[i:i + 2] for i in range(0, len(lst), 2)]) # chunk into pairs
print(max(set(lst), key=lst.count)) # most frequent (O(n^2), fine for small)
print(list(zip(*[1, 2], [3, 4]))) # transpose [(1,3), (2,4)]
print(sum(1 for x in lst if x > 1)) # count matching
print(next((x for x in lst if x > 2), None)) # first match or None
print(dict(zip("abc", range(3)))) # build dict from two lists
print({k: v for k, v in {"a": 1, "b": 0}.items() if v}) # filter dict
print(" ".join(map(str, lst))) # numbers -> string
print(list(map(int, "1 2 3".split()))) # string -> numbers
print(lst[::-1], lst[-3:], lst[:-1]) # reverse / last 3 / all but last
a, b = 5, 7; a, b = b, a # swap
print(bin(10).count("1")) # set bits
print(divmod(3725, 60)) # (62, 5) minutes and seconds

```

19. Coding Interview Patterns (Medium Level)

19.1 Hash map, counting, grouping

```

from collections import defaultdict, Counter

def two_sum(nums, target): # O(n): store complement seen so far
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return []
print(two_sum([2, 7, 11, 15], 9)) # [0, 1]

def group_anagrams(words):
    g = defaultdict(list)
    for w in words:
        g["".join(sorted(w))].append(w)
    return list(g.values())
print(group_anagrams(["eat", "tea", "tan", "ate", "nat"])) # [['eat', 'tea', 'ate'], ['tan', 'nat']]

def top_k_frequent(nums, k):
    return [n for n, _ in Counter(nums).most_common(k)]
print(top_k_frequent([1, 1, 1, 2, 2, 3], 2)) # [1, 2]

def has_duplicate(nums): return len(nums) != len(set(nums))
def first_duplicate(nums):
    seen = set()
    for n in nums:
        if n in seen: return n
        seen.add(n)
def longest_consecutive(nums):
    s, best = set(nums), 0
    for n in s:
        if n - 1 not in s: # start of a run

```

```

        m = n
        while m + 1 in s: m += 1
        best = max(best, m - n + 1)
    return best
print(longest_consecutive([100, 4, 200, 1, 3, 2])) # 4

```

19.2 Two pointers and sliding window

```

def reverse_in_place(a):
    l, r = 0, len(a) - 1
    while l < r:
        a[l], a[r] = a[r], a[l]; l += 1; r -= 1
    return a

def pair_sum_sorted(a, target): # sorted input, O(n)
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == target: return (a[l], a[r])
        l, r = (l + 1, r) if s < target else (l, r - 1)

def remove_dupes_sorted(a): # in place, returns new length
    if not a: return 0
    w = 1
    for i in range(1, len(a)):
        if a[i] != a[i - 1]:
            a[w] = a[i]; w += 1
    return w

def max_sum_window(a, k): # fixed window
    cur = best = sum(a[:k])
    for i in range(k, len(a)):
        cur += a[i] - a[i - k]
        best = max(best, cur)
    return best
print(max_sum_window([2, 1, 5, 1, 3, 2], 3)) # 9

def longest_unique_substring(s): # variable window
    last, start, best = {}, 0, 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1
        last[ch] = i
        best = max(best, i - start + 1)
    return best
print(longest_unique_substring("abcabcbb")) # 3

def kadane(a): # max subarray sum
    cur = best = a[0]
    for x in a[1:]:
        cur = max(x, cur + x); best = max(best, cur)
    return best
print(kadane([-2, 1, -3, 4, -1, 2, 1, -5, 4])) # 6

```

19.3 Sorting, searching, intervals, matrices

```

def binary_search(a, target): # sorted list, O(log n)
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == target: return mid
        if a[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
print(binary_search([1, 3, 5, 7, 9], 7)) # 3

def merge_intervals(iv):
    out = []
    for s, e in sorted(iv):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)

```

```

        else:
            out.append([s, e])
        return out
print(merge_intervals([[1, 3], [2, 6], [8, 10], [15, 18]])) # [[1, 6], [8, 10], [15, 18]]

def merge_sorted(a, b):
    i = j = 0; out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]: out.append(a[i]); i += 1
        else: out.append(b[j]); j += 1
    return out + a[i:] + b[j:]

def rotate_matrix(m):
    # 90 degrees clockwise
    return [list(r) for r in zip(*m[::-1])]
print(rotate_matrix([[1, 2], [3, 4]])) # [[3, 1], [4, 2]]

def rotate_list(a, k):
    k %= len(a); return a[-k:] + a[:-k]
print(rotate_list([1, 2, 3, 4, 5], 2)) # [4, 5, 1, 2, 3]

def quicksort(a):
    if len(a) < 2: return a
    p = a[len(a) // 2]
    return quicksort([x for x in a if x < p]) + [x for x in a if x == p] + quicksort([x for x in a if x > p])
print(sorted([3, 1, 2], key=lambda x: -x)) # custom order

```

19.4 Stack, queue, linked list, tree, graph

```

from collections import deque

def valid_parentheses(s):
    pairs, stack = {}, {}
    pairs["("] = ")", "["] = "]", "{"] = "}"
    for ch in s:
        if ch in pairs.values(): stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]: return False
    return not stack
print(valid_parentheses("([()])"), valid_parentheses("(")) # True False

class Node:
    def __init__(self, val, next=None): self.val, self.next = val, next
def reverse_list(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev
def to_list(head):
    out = []
    while head: out.append(head.val); head = head.next
    return out
print(to_list(reverse_list(Node(1, Node(2, Node(3))))) # [3, 2, 1]

class TreeNode:
    def __init__(self, v, l=None, r=None): self.v, self.l, self.r = v, l, r
def inorder(t): return inorder(t.l) + [t.v] + inorder(t.r) if t else []
def height(t): return 1 + max(height(t.l), height(t.r)) if t else 0
def level_order(t):
    out, q = [], deque([t] if t else [])
    while q:
        level = []
        for _ in range(len(q)):
            n = q.popleft(); level.append(n.v)
            q.extend(x for x in (n.l, n.r) if x)
        out.append(level)
    return out
tree = TreeNode(2, TreeNode(1), TreeNode(3))
print(inorder(tree), height(tree), level_order(tree)) # [1,2,3] 2 [[2],[1,3]]

graph = {"A": ["B", "C"], "B": ["D"], "C": ["D"], "D": []}
def bfs(g, start):
    seen, q, order = {start}, deque([start]), []
    while q:
        n = q.popleft(); order.append(n)

```

```

        for nb in g[n]:
            if nb not in seen: seen.add(nb); q.append(nb)
    return order
def dfs(g, n, seen=None):
    seen = seen if seen is not None else set()
    seen.add(n)
    for nb in g[n]:
        if nb not in seen: dfs(g, nb, seen)
    return seen
print(bfs(graph, "A"), sorted(dfs(graph, "A"))) # ['A', 'B', 'C', 'D'] ['A', 'B', 'C', 'D']

```

19.5 Recursion, backtracking, dynamic programming

```

def permutations(items):
    if len(items) <= 1: return [items]
    out = []
    for i, x in enumerate(items):
        for p in permutations(items[:i] + items[i + 1:]):
            out.append([x] + p)
    return out
print(len(permutations([1, 2, 3]))) # 6

def subsets(nums):
    res = [[]]
    for n in nums:
        res += [r + [n] for r in res]
    return res
print(subsets([1, 2])) # [[], [1], [2], [1, 2]]

def climb_stairs(n): # DP: ways to climb 1 or 2 steps
    a, b = 1, 1
    for _ in range(n - 1): a, b = b, a + b
    return b
print(climb_stairs(5)) # 8

def coin_change(coins, amount): # min coins, bottom-up DP
    dp = [0] + [float("inf")] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a: dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != float("inf") else -1
print(coin_change([1, 2, 5], 11)) # 3

def lcs(a, b): # longest common subsequence length
    dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            dp[i][j] = dp[i-1][j-1] + 1 if a[i-1] == b[j-1] else max(dp[i-1][j], dp[i][j-1])
    return dp[-1][-1]
print(lcs("abcde", "ace")) # 3

```

19.6 Design a class: LRU cache and rate limiter

```

from collections import OrderedDict
import time

class LRUCache:
    def __init__(self, capacity):
        self.cap, self.d = capacity, OrderedDict()
    def get(self, key):
        if key not in self.d: return -1
        self.d.move_to_end(key) # mark most recently used
        return self.d[key]
    def put(self, key, val):
        self.d[key] = val; self.d.move_to_end(key)
        if len(self.d) > self.cap:
            self.d.popitem(last=False) # evict least recently used
c = LRUCache(2); c.put(1, "a"); c.put(2, "b"); c.get(1); c.put(3, "c")
print(c.get(2), c.get(1)) # -1 a

class RateLimiter: # sliding window: max n calls per period seconds

```

```

def __init__(self, n, period):
    self.n, self.period, self.calls = n, period, []
def allow(self):
    now = time.monotonic()
    self.calls = [t for t in self.calls if now - t < self.period]
    if len(self.calls) < self.n:
        self.calls.append(now); return True
    return False

```

19.7 Data-engineering style coding questions

```

import csv, json, re
from collections import Counter, defaultdict

# 1) Parse log lines and count status codes
logs = ['2026-09-24 10:00:01 GET /a 200', '2026-09-24 10:00:02 GET /b 500', '2026-09-24 10:00:03 POST /a 200']
codes = Counter(line.split()[-1] for line in logs)
print(codes) # Counter({'200': 2, '500': 1})
pat = re.compile(r"^(S+ S+) (\w+) (S+) (\d{3})$")
parsed = [m.groups() for line in logs if (m := pat.match(line))]

# 2) Running total / moving average
def running_total(xs):
    t = 0
    for x in xs:
        t += x; yield t
print(list(running_total([1, 2, 3, 4]))) # [1, 3, 6, 10]
def moving_avg(xs, k):
    return [sum(xs[i:i + k]) / k for i in range(len(xs) - k + 1)]

# 3) Merge two datasets on a key (join) in pure Python
left = [{"id": 1, "a": "x"}, {"id": 2, "a": "y"}]
right = [{"id": 1, "b": 10}, {"id": 3, "b": 30}]
ridx = {r["id"]: r for r in right}
inner = [{"*l, **ridx[l["id"]]} for l in left if l["id"] in ridx]
left_join = [{"*l, **ridx.get(l["id"], {"b": None})} for l in left]

# 4) Latest record per key (dedupe by max timestamp)
recs = [{"id": 1, "ts": 1, "v": "old"}, {"id": 1, "ts": 5, "v": "new"}, {"id": 2, "ts": 2, "v": "z"}]
latest = {}
for r in recs:
    if r["id"] not in latest or r["ts"] > latest[r["id"]]["ts"]:
        latest[r["id"]] = r
print(list(latest.values()))

# 5) Find the differences between two snapshots (added / removed / changed)
old = {1: "a", 2: "b", 3: "c"}; new = {2: "b", 3: "c", 4: "d"}
added = new.keys() - old.keys()
removed = old.keys() - new.keys()
changed = {k for k in old.keys() & new.keys() if old[k] != new[k]}
print(added, removed, changed) # {4} {1} {3}

# 6) CSV to JSON with type conversion and validation
def csv_to_json(src, dst):
    out = []
    with open(src, newline="", encoding="utf-8") as f:
        for i, row in enumerate(csv.DictReader(f), start=2): # line 1 is header
            try:
                out.append({"id": int(row["id"]), "amt": float(row["amt"])})
            except (ValueError, KeyError) as e:
                print(f"skip line {i}: {e}")
    with open(dst, "w", encoding="utf-8") as f:
        json.dump(out, f, indent=2)

# 7) Explode delimited column, unique values, and null handling
row = {"id": 1, "tags": "a|b|c"}
exploded = [{"id": row["id"], "tag": t} for t in row["tags"].split("|")]

# 8) Sessionise events: new session if gap > 30 min
def sessionize(ts_sorted, gap=1800):
    sessions, cur = [], [ts_sorted[0]]
    for t in ts_sorted[1:]:
        if t - cur[-1] > gap: sessions.append(cur); cur = []
    cur.append(t)

```

```

sessions.append(cur); return sessions
print(sessionize([0, 100, 5000, 5100])) # [[0, 100], [5000, 5100]]

```

19.8 Big-O quick reference

Complexity	Name	Typical example
O(1)	constant	dict lookup, list index, append
O(log n)	logarithmic	binary search, heap push/pop
O(n)	linear	single loop, in on list, sum
O(n log n)	linearithmic	sorted , merge sort
O(n ²)	quadratic	nested loops, naive duplicate check
O(2 ⁿ)	exponential	naive Fibonacci, subsets
O(n!)	factorial	permutations

Approach checklist for any problem: restate the problem, ask about edge cases (empty, one item, duplicates, negatives, huge input), give the brute force first, then improve with a hash map, sorting, two pointers, or a heap; state time and space complexity; test with a tiny example out loud.

20. Interview Questions and Short Answers

20.1 Python fundamentals

Question	Answer
List vs tuple?	List is mutable, tuple immutable and hashable (usable as dict key), slightly faster/smaller.
List vs set vs dict?	Ordered sequence / unique unordered items / key-value mapping.
Mutable vs immutable?	Can the object change in place. Mutable: list, dict, set. Immutable: int, str, tuple, frozenset.
is vs == ?	Identity (same object) vs equality (same value).
Shallow vs deep copy?	Shallow copies the outer container only; deep copies nested objects recursively (copy.deepcopy).
What is a decorator?	A callable that wraps another function to add behaviour, applied with @name .
Generator vs list?	Generator yields lazily one item at a time (O(1) memory), single pass.
Iterator vs iterable?	Iterable can produce an iterator (__iter__); iterator has __next__ and holds state.
What is the GIL?	A CPython lock so only one thread runs bytecode at a time; use multiprocessing for CPU-bound work.
*args / **kwargs ?	Collect extra positional args into a tuple / keyword args into a dict.
Default argument gotcha?	Defaults are evaluated once at definition; never use a mutable default, use None .
__name__ == "__main__" ?	True when run as a script, False when imported; guards entry point code.
Pass by value or reference?	Pass by object reference (assignment): mutating a mutable argument is visible to the caller, rebinding is not.
What are LEGB rules?	Local, Enclosing, Global, Built-in name lookup.
How is memory managed?	Reference counting plus a cyclic garbage collector; private heap.
del vs remove vs pop ?	del by index/name, remove by value, pop by index and returns the item.
append vs extend ?	append adds one object (even a list); extend adds each element of an iterable.
sort() vs sorted() ?	In place returning None vs returns a new list.
List comprehension vs map ?	Comprehension is usually clearer; both are eager (map is lazy iterator in py3).
What is a lambda?	Anonymous single-expression function.
What are pass , break , continue ?	No-op placeholder, exit loop, skip to next iteration.
What is __init__.py ?	Marks a directory as a package (optional for namespace packages since 3.3).
Module vs package?	A .py file vs a directory of modules.

Question	Answer
What is a virtual environment?	Isolated interpreter and packages per project (venv).
What is PEP 8?	The Python style guide.
What are f-strings?	Formatted string literals evaluated at runtime, f"{x:.2f}" .
range in Python 3?	Lazy immutable sequence, O(1) memory.
Is Python compiled or interpreted?	Compiled to bytecode then executed by the CPython VM.
What are dunder methods?	Special methods like __init__ , __len__ that customise object behaviour.
Explain with .	Context manager guarantees setup/cleanup via __enter__ / __exit__ .
What is monkey patching?	Changing classes/modules at runtime; risky, mostly for tests.
What is global vs nonlocal ?	Rebind a module-level name vs an enclosing function's name.
Threading vs multiprocessing vs asyncio?	I/O-bound threads, CPU-bound processes, many network waits on one thread.
How do you handle big files?	Iterate lazily, chunk, use generators, pandas chunksize , avoid read() .
How do you make code testable?	Small functions, dependency injection, no globals, main(argv) , pytest + mocks.

20.2 Data engineering and API scenario questions

Scenario	Strong answer outline
Consume a paginated API	Identify pagination style, loop with a generator, cap pages, timeouts, retry with backoff on 429/5xx, respect Retry-After , log counts, checkpoint cursor.
Make a pipeline idempotent	Deterministic keys, MERGE/upsert or delete-insert per partition, watermark advanced only after success, staging tables.
Handle bad records	Try/except per record, quarantine to a reject file with the error, fail only above a threshold, log counts.
Handle schema drift	Explicit schema/column mapping, validate expected columns, default missing, alert on new columns, keep raw layer.
Load 100 GB CSV	Stream/chunk, compress, split files, parallelise, use warehouse bulk load (stage + COPY), validate counts.
Secure credentials	Environment variables or secrets manager, never in code/logs, least privilege roles, rotate keys.
Speed up slow Python ETL	Profile (cProfile), vectorise with pandas, avoid row-wise loops, batch DB calls, threads for I/O, processes for CPU, push work into the database.
Validate a migration	Row counts, per-column aggregates, checksums, null/distinct counts, sample row diffs, reconcile by key.
Log for a production job	Structured logs with run_id, counts and durations, levels, rotation, alerts on ERROR, exit non-zero on failure.
Retry safely	Only idempotent operations, exponential backoff with jitter, max attempts, distinguish transient vs permanent errors.

21. Completeness Check and What To Add Next

21.1 What this guide covers

Core syntax, operators, casting, strings, all built-in data structures with methods and complexity, `collections` / `heapq` / `bisect`, control flow with `match`, functions with `*args` / `**kwargs`, closures, decorators, generators, context managers, exception handling and custom exceptions, file read / write / edit patterns (text, CSV, JSON, JSONL, binary, gzip), `os` / `sys` / `pathlib` / `shutil` / `subprocess`, `argparse`, `logging`, full OOP (classes, inheritance, MRO, ABCs, dunder methods, dataclasses, patterns), REST APIs with authentication, retries and four pagination styles, pandas, DB-API, a full DuckDB chapter (connections, file queries, SQL features, UDFs, reconciliation), Snowflake connector, migration and validation patterns, concurrency, testing, plus coding-pattern problems and rapid-fire Q&A.

21.2 Verdict for Python data engineering work

This is enough for **Python coding rounds at medium level** and for the Python portion of data engineering interviews. Strong data engineering candidates are usually also tested on the items below, which are outside a pure Python guide. Treat this list as your study backlog.

Topic to add	Why it matters	Where to practise
SQL (joins, window functions, CTEs, MERGE, dedupe, gaps and islands)	Almost always tested alongside Python	Write every pandas example also in SQL using DuckDB (chapter 17)
PySpark / DataFrame API (transformations vs actions, partitions, joins, shuffles, UDF cost)	Large-scale processing	<code>df.groupBy().agg()</code> , <code>Window</code> , broadcast joins
Orchestration (Airflow DAGs, retries, backfills, sensors, idempotent tasks)	Scheduling pipelines	Small DAG with 3 tasks and retries
Data modelling (star schema, SCD types, normalisation, data vault)	Design rounds	Model an orders domain
Cloud storage and file formats (S3/ADLS/GCS, Parquet vs CSV vs Avro, partitioning, compression)	Real pipelines use columnar files	<code>df.to_parquet(partition_cols=[...])</code>
Data quality frameworks (Great Expectations, dbt tests)	Reliability	Write checks like section 16.4
dbt and ELT thinking	Modern warehouse workflows	Models, tests, incremental materialisation
Streaming basics (Kafka topics, offsets, at-least-once vs exactly-once)	Real-time pipelines	Consumer loop with commit
Pydantic / schema validation	Validating API payloads	<code>BaseModel</code> with typed fields
Docker, Git, CI basics	Everyday engineering	Dockerfile for an ETL script
Performance profiling (<code>cProfile</code> , <code>timeit</code> , <code>memory_profiler</code>)	Scaling answers	Profile a slow loop and fix it
Warehouse specifics (Snowflake: stages, COPY INTO, Snowpipe, streams and tasks, clustering, zero-copy clone)	Migration and platform roles	Use the connector examples in 16.3

21.3 Study plan and last-minute checklist

- ? Day 1-2: sections 1-6 (types, casting, operators, structures, control flow); run every snippet in a REPL and change inputs.
- ? Day 3: functions, `*args` / `**kwargs`, closures, decorators, generators, exceptions.
- ? Day 4: files, `os` / `sys`, `argparse`, `logging`; build one small CLI that reads a CSV, validates rows, logs counts and writes JSON.
- ? Day 5: OOP; re-implement the inventory example and an abstract `Reader` with two subclasses.
- ? Day 6: APIs and pagination; write all four pagination generators against a fake API.
- ? Day 7: sections 19-20 problems timed (20 minutes each), then explain solutions aloud.

Before the interview: can you (1) explain mutable vs immutable with an example, (2) write a decorator from memory, (3) write a paginated fetch with retries, (4) read a large file lazily, (5) parse args with `argparse`, (6) set up logging with a file handler, (7) write a class with `__init__`, `__repr__`, property and inheritance, (8) name the complexity of your solution? If yes to all eight, you are ready for medium-level Python rounds.